A Probabilistic Process Calculus for Durable Multi-Agent LLM Systems

Aniketh Bandlamudi
University of Virginia
zpq3uh@virginia.edu me@aniketh.net

November 2026

Abstract

Multi-agent systems built on large language models (LLMs) fail often: across seven open-source multi-agent frameworks, Cemri et al. [6] measure failure rates between 41% and 86.7%, with many failures arising between agents rather than within one. Formal models of LLM agents have begun to address this, but each covers only part of the problem. LLMbda [12] gives a probabilistic, machine-checked semantics for a single agent; the MSC coordination calculus [4] proves coordination properties of many agents but treats the model as a black box whose outputs are arbitrary; and durable-execution engines such as Temporal recover crashed workflows by recording each nondeterministic step as an opaque value, without a semantics for its distribution.

We present *Agentical*, a probabilistic process calculus for multi-agent LLM systems in which each LLM call is a draw from an abstract probability measure and every externally determined value is recorded in an append-only journal. The calculus has six primitives (**Agent**, **Oracle**, **Chan**, **Mem**, **Journal**, **Sup**) and a small-step semantics with separate record and replay modes that keeps probabilistic choice (oracle draws) apart from adversarial choice (scheduling). We prove that replaying a run from its journal reproduces its observable trace with probability 1 and that record and replay are a coupling of the same randomness, including recovery from a crash at a consistent cut (T1). We also give a probabilistic noninterference theorem for concurrent agents under a floating-label discipline (T2) and elementary compositional probability bounds (T3), with proof sketches for T2 and T3. The single-agent core of T1 is mechanized in Lean 4, and a dependency-free Rust prototype implements single-agent record, replay, and write-ahead-logged effects; on it we measure replay determinism, recovery from a crash at every journal position, and journaling overhead.

Contents

1 Introduction **4**

2 Background **6**
2.1 Process Calculi 6
2.2 Probabilistic Operational Semantics 6
2.3 Durable Execution 7
2.4 Positioning Summary 7

3 The Agentical Calculus: Six Primitives **8**
3.1 Overview 8
3.2 Surface Syntax 8

3.3 Primitive 1: Agent . . . 9
3.4 Primitive 2: Oracle . . . 10
3.5 Primitive 3: Channel . . . 10
3.6 Primitive 4: Memory . . . 10
3.7 Primitive 5: Journal . . . 11
3.8 Primitive 6: Supervisor . . . 11
3.9 Syntactic Sugar and Derived Forms . . . 12
3.10 Summary . . . 12

4 Operational Semantics **12**
4.1 Syntax . . . 12
4.2 Configurations . . . 13
4.3 Structural Congruence . . . 14
4.4 Observables and Traces . . . 15
4.5 Reduction Rules . . . 15
4.6 Probabilistic Transition System . . . 18
4.7 Well-Labelled Journals . . . 19

5 Metatheory **20**
5.1 Supporting Lemmas . . . 20
5.2 T1: Replay Soundness Under Nondeterminism . . . 22
5.2.1 The coupling and the $\hat{\mathfrak{s}} \to \mathfrak{s}$ handover . . . 23
5.2.2 Corollaries . . . 24
5.3 T2: Multi-Agent Probabilistic Noninterference . . . 25
5.4 T3: Compositional Probability Bounds . . . 27
5.5 Assumptions Ledger . . . 27

6 Applications **28**
6.1 Tool Use with Retries and Crash Recovery . . . 28
6.2 Map-Reduce Summarization . . . 29
6.3 Debate with Confidential Sources . . . 30
6.4 Hierarchical Delegation with Supervision . . . 30

7 Evaluation **31**
7.1 E1: Replay Determinism . . . 32
7.2 E2: Crash Recovery at Every Journal Position . . . 32
7.3 E3: Journaling Overhead . . . 32
7.4 Threats to Validity . . . 33

8 Related Work **33**
8.1 Formal Models of LLM Agents . . . 34
8.2 Probabilistic Semantics . . . 35
8.3 Information-Flow Control for Concurrent Programs . . . 35
8.4 Rollback Recovery and Durable Execution . . . 35
8.5 Summary . . . 36

9 Discussion **36**
9.1 The Rust Prototype 36
9.2 Limitations 37
9.3 Open Problems 38

10 Conclusion **39**

Acknowledgments and Use of AI Tools **39**

A Notation **42**

1 Introduction

The problem

Large language models are increasingly deployed as autonomous agents that plan, call tools, delegate to sub-agents, and keep long-running state. Their failures are frequent and often arise in the interaction between agents rather than in any single model call: Cemri et al. [6] annotate more than 1,600 execution traces from seven open-source multi-agent frameworks, measure failure rates between 41% and 86.7%, and find that a large share of failures fall into inter-agent misalignment and task verification. Such systems are also hard to make durable. An agent run may last minutes or hours, call external tools with real side effects, and crash part-way through; restarting it from scratch re-samples every model call and may repeat effects that already happened.

Formal models of these systems are appearing, but each leaves out part of this picture.

Single-agent calculi. LLMbda [12] is an untyped call-by-value λ-calculus with conversations, code generation, and dynamic information-flow control. Its first version assumed, following FIDES [9], that the underlying model is deterministic; the current version gives a probabilistic semantics and proves a termination-insensitive probabilistic noninterference theorem, with all proofs in Lean. LLMbda studies one agent's program, however: it does not model concurrently running agents, schedulers, or crash recovery. Other recent calculi are likewise sequential: λ_A [26] adds oracle calls and probabilistic choice to a typed λ-calculus and proves type safety and termination in Coq, and PPDL [27] gives LLM-based flows a semantics as probabilistic programs.

Black-box coordination. The MSC coordination calculus [4] specifies multi-agent workflows as message sequence charts, projects them to deadlock-free local programs, and is machine-checked in Lean 4 with an open-source implementation, ZipperGen. By design it establishes coordination properties independently of LLM nondeterminism: model outputs are treated as arbitrary. This is robust, but it cannot express quantitative statements such as "the pipeline succeeds with probability at least 0.99", which require reasoning about the distribution of outputs.

Deterministic replay. Durable-execution engines, exemplified by Temporal [38], make long-running workflows crash-recoverable by replaying an event history. Workflow code must be deterministic; calls to the outside world, including LLM calls, run as *Activities* whose results are recorded once and returned from the history on replay. The engine therefore treats each LLM call as an opaque recorded value: it offers no way to state or check properties of the distribution the value was drawn from. The underlying idea, logging every nondeterministic event so that a piecewise-deterministic process can be replayed after a failure, is classical in rollback-recovery for message-passing systems [11, 37]. Recent tools for LLM agents record and replay runs at their nondeterministic boundaries [8, 29] or check resume behaviour against machine-checked contracts [19, 43], and Wu et al. [41] show that agent rollback can restore states that are not consistent causal cuts, the failure that the consistent-cut recovery of Section 5 rules out. None of these gives a semantics for the distribution of model outputs. Recording is necessary in practice: Khatchadourian [20] reports, for repeated executions of tool-using financial agents, a configuration that repeated its recorded decision on 84% of cases but its ordered sequence of tool calls on only 26%.

Our contribution

We present *Agentical*, a probabilistic process calculus that combines these concerns: it models many concurrently running agents, treats each LLM call as a draw from a probability measure, and gives crash recovery a semantics.

1. **A probabilistic semantics for multi-agent LLM systems with a journal (Sections 3–4).** The calculus has six primitives: **Agent**, **Oracle**, **Chan**, **Mem**, **Journal**, **Sup**. An **Oracle** call is a draw from an abstract measure μ_e over results, so a system denotes a probabilistic automaton in which scheduling (adversarial) and oracle draws (probabilistic) are kept separate. The **Journal** records every externally determined value, and the semantics has a record mode and a replay mode; durable effects follow a three-phase write-ahead discipline.

2. **Replay soundness under nondeterminism (T1, Section 5.2).** Replaying a run from its journal reproduces the recorded trace with probability 1, and the record and replay runs are a coupling of the same oracle randomness, so every trace-measurable event has the same probability under the record law conditioned on the journal as under replay. For concurrent agents, recovery from a crash at a consistent cut re-delivers exactly the messages that were not yet received. Logging nondeterministic events for replay is not new; what is new is stating and proving its soundness inside a probabilistic semantics for LLM agents. The single-agent, oracle-only core of T1 is mechanized in Lean 4 (Section 9.3).

3. **Noninterference and compositional bounds for concurrent agents (T2–T3, Sections 5.3–5.4).** Under a floating-label discipline and a scheduler that cannot observe High steps, the distribution of Low-observable traces does not depend on High data (T2); this extends probabilistic noninterference from a single agent to concurrently scheduled agents. Local per-agent probability contracts imply global bounds by Boole's inequality, or by a product bound under an explicit independence assumption (T3). We give proof sketches for T2 and T3; neither is mechanized.

4. **A prototype and measurements (Sections 7 and 9.1).** A dependency-free Rust crate defines one interface per primitive and implements single-agent record, replay, crash resume, and write-ahead-logged effects. We measure replay determinism, recovery from a crash at every journal position, and journaling overhead on it.

The paper's source, the Lean development, and the Rust prototype with all experiment harnesses are available at `https://github.com/ankthba/agentical`.

Scope

We do not claim to invent probabilistic operational semantics: probabilistic automata [34], the probabilistic π-calculus [15], PEPA [16], and the probabilistic λ-calculus [5] are established ancestors. We do not claim to be first to formalize LLM agents or their coordination (Section 8), nor to invent event logging for replay. Our claim is the combination: a probabilistic operational semantics for concurrently running LLM agents in which durable replay is a theorem. The prototype covers a single agent; multi-agent execution is specified and proved but not yet implemented (Section 9.2). Table 1 compares capabilities, and Appendix A collects the notation.

Roadmap

Section 2 reviews process calculi, probabilistic semantics, and durable execution. Section 3 introduces the six primitives informally, and Section 4 gives the probabilistic operational semantics in record and replay modes. Section 5 states and proves T1–T3. Section 6 applies the calculus to four patterns (tool use with crash recovery, map-reduce, a debate with confidential sources, and hierarchical delegation). Section 7 reports experiments on the Rust prototype. Section 8 discusses related work, Section 9 the prototype, limitations, and open problems, and Section 10 concludes.

2 Background

This section reviews the three research traditions our work synthesizes: process calculi for concurrent systems, probabilistic operational semantics, and durable-execution models.

2.1 Process Calculi

Process calculi provide compositional syntax and operational semantics for communicating concurrent systems. The π-calculus [28] extends CCS with mobile channel names, enabling channels themselves to be communicated and thereby supporting dynamic process creation and delegation. Key features relevant to us are (1) a small-step reduction relation over configurations, (2) parallel composition $P \mid Q$ as a first-class operator, and (3) restriction $\nu c.\, P$ for fresh private names. Session types [17, 18] extend the π-calculus with interaction type disciplines, protocols specifying the sequence and types of messages a channel must carry, enabling static enforcement of communication contracts such as deadlock freedom.

LLMbda [12] brings programming-language semantics to LLM agents. It is an untyped call-by-value λ-calculus with conversation primitives, code generation, and dynamic information-flow control (IFC) in which every value carries a label. Its first version assumed the underlying model to be deterministic; the current version gives a probabilistic semantics and proves a termination-insensitive probabilistic noninterference theorem, with all proofs in Lean. LLMbda models a single agent's program; it does not address concurrently scheduled agents or crash recovery, which are the focus of this paper.

Schlapbach [33] applies process-calculus tools to agentic tool protocols such as the Model Context Protocol; the second version of that preprint was withdrawn for a logical flaw in one theorem, and a revised version has since appeared.

The MSC coordination calculus [4] uses message sequence charts to specify global coordination protocols for multi-agent LLM systems, projects them to local agent programs, and proves deadlock-freedom. Results are machine-checked in Lean 4, and a Python implementation (ZipperGen) is provided. Its nondeterminism handling is the key contrast with our work: the MSC calculus proves coordination properties *independently* of LLM outputs, treating the LLM as an arbitrary black box. This is a valuable approach but it cannot express quantitative bounds about the distribution of outcomes.

2.2 Probabilistic Operational Semantics

Probabilistic operational semantics augments the standard labelled transition relation with probability measures, enabling statements about distributions over executions.

Probabilistic automata. Segala and Lynch [34] define *probabilistic automata* as a model with both nondeterministic (adversarial) and probabilistic transitions. The key design choice, which we adopt, is to keep the two layers separate: a *scheduler* resolves nondeterministic choices (e.g., message orderings), and once scheduling is fixed, the remaining randomness is governed by a probability measure. Properties are then stated as: for *all* schedulers, the probability of some event is at least p. This two-layer discipline prevents the classic conflation error of treating probabilistic and nondeterministic transitions as the same kind of thing.

Probabilistic process calculi. The probabilistic π-calculus of Herescu and Palamidessi [15] extends the π-calculus with probabilistic choice, giving a measure-theoretic operational semantics for mobile communicating processes. PEPA [16] is a stochastic process algebra oriented toward performance modelling, with rates governing transitions. These works establish the technical toolkit, sub-probability distributions over configurations, congruence properties, bisimilarity up to probability, that our semantics builds on.

Probabilistic λ-calculus. Borgström et al. [5] give distribution-based big-step and small-step semantics for a probabilistic call-by-value λ-calculus and prove them equivalent. The first version of LLMbda named this work as the route to a probabilistic semantics, which its current version takes. Our **Oracle** primitive can be understood as inlining a single application of this semantics at each LLM call site.

2.3 Durable Execution

Durable execution systems, Temporal [38] being the canonical production example, give long-running workflows crash-recoverable semantics. The mechanism is an *event history*: an append-only log of all externally-determined values received by the workflow. On restart, the workflow code is re-executed from the beginning, but any call that would normally block on an external response instead returns the stored value from the log immediately, a technique called *deterministic replay*.

The operative constraint is that *workflow code must be deterministic*: if the same code is re-run against the same event history, it must make the same branching decisions. Calls to the outside world (HTTP requests, LLM calls) run as *Activities*, whose results are recorded once and thereafter returned from the history, and other nondeterministic values such as random numbers are obtained through replay-safe SDK operations that record them [39]. This design makes replay correct, but it gives no semantics to the *distribution* of Activity outcomes, only to the single realized one. It cannot, for instance, reason about whether a retry-until-success strategy terminates in expectation, or what the probability is that a compensation saga completes successfully.

Logging nondeterministic events for replay is itself classical in rollback recovery for message-passing systems [11, 37]. Our Journal primitive (Section 3) applies the same idea within a calculus that treats each recorded outcome as a draw from a named distribution, enabling replay soundness to be stated and proved as a theorem rather than assumed as a design invariant.

2.4 Positioning Summary

Table 1 compares Agentical with the closest related systems; Section 8 discusses them in detail.

Table 1: Comparison of formal approaches to LLM agent systems. ✓ = supported, × = not supported, ∼ = partial.

System	Multi-agent	Prob. oper. sem.	Durable	Mechanized	Impl.
LLMbda [12]	×	✓	×	✓ (Lean)	✓ (verified interpreter)
MCP calculus [33]	×	×	×	×	×
MSC coord. [4]	✓	× (black box)	∼ (ZipperGen, informal)	✓ (Lean 4)	✓ (Python)
Temporal [38]	∼	× (det. required)	✓	×	✓
Agentical (this work)	✓	✓	✓	∼ (single-agent T1)	∼ (Rust, single agent)

3 The Agentical Calculus: Six Primitives

3.1 Overview

The Agentical calculus is built from six primitives. Each primitive must be simultaneously (a) a syntactic category, (b) a component of the operational semantics, (c) a Rust trait, and (d) a carrier of at least one formal property. We call this the *four-column discipline*: anything that satisfies only two or three columns is an implementation convenience, not a primitive.

Two primitives are novel: **Oracle** (an LLM invocation modelled as a measurable distribution) and **Journal** (an append-only event history that enables provably sound replay under nondeterminism). The remaining four (**Agent**, **Chan**, **Mem**, **Sup**) adapt well-established models, π-calculus, actors, IFC stores, saga/supervisor trees, to make the novel two composable.

3.2 Surface Syntax

This section introduces the primitives informally, using a lightweight surface syntax chosen for readability. Section 4 fixes the core syntax and the five-component configuration on which all rules and proofs are stated; the surface forms below abbreviate core terms (for example, $x \leftarrow \mathbf{oracle}(t);\ B$ stands for $\mathbf{let}\ x \Leftarrow \mathbf{oracle}[\ell](t)\ \mathbf{in}\ M$, and $\mathbf{delegate}(B_1, c);\ x \leftarrow c?;\ B_2$ for a **spawn** followed by a receive on c).

Values and terms

Let $\mathcal{N}$ be a countably infinite set of *names* (channel names, agent identifiers, memory keys). Let $\mathcal{L}$ be a finite lattice of *IFC labels*, with partial order $\sqsubseteq$ and join $\sqcup$. We write $a, b, c, \ldots$ for channel names and x, y, z for variables.

Definition 3.1 (Surface terms)**.** Terms t, u, v and behaviours B are defined by the following grammar.

$$
\begin{array}{lll}
v & ::= () & \text{unit} \\
& \mid n & \text{name } (n \in \mathcal{N}) \\
& \mid \lambda x.\, B & \text{abstraction}
\end{array}
$$

B	$::= \mathbf{0}$	inert
	$\mid x \leftarrow \mathbf{oracle}(t);\ B$	oracle draw
	$\mid c!v;\ B$	channel send
	$\mid x \leftarrow c?;\ B$	channel receive
	$\mid \mathbf{write}(k^{\ell}, v);\ B$	memory write (label ℓ)
	$\mid x \leftarrow \mathbf{read}(k^{\ell});\ B$	memory read
	$\mid \mathbf{spawn}(B_1 \parallel B_2)$	parallel composition
	$\mid \nu c.\ B$	fresh channel restriction
	$\mid \mathbf{delegate}(B_1, c);\ x \leftarrow c?;\ B_2$	spawn sub-agent, await result on c
	$\mid \mathbf{fault}(f)$	raise fault f

Remark 3.2. The sequencing $x \leftarrow \mathbf{oracle}(t);\ B$ is *monadic*: it binds the realized outcome x in the continuation B. This makes the dependence of downstream computation on the sampled value syntactically explicit, which is essential for the Journal to record the right events.

Configurations

Definition 3.3 (Configuration, informal view)**.** Informally, a running system consists of

$$\langle \mathcal{A},\ \mu,\ \mathcal{J},\ \mathfrak{s} \rangle$$

where:

- $\mathcal{A}$ is a finite pool of *agent instances* $A_i = \langle id_i, \Sigma_i, B_i \rangle$ (identity, private state, behaviour),
- μ is the current *oracle measure* assignment (an abstract probability kernel from prompts to result terms),
- $\mathcal{J}$ is the current *journal state* (Definition 3.11), and
- $\mathfrak{s}$ is the current *scheduler state* (an element of a scheduler algebra that resolves Channel interleaving; Definition 3.8).

In the formal semantics (Definition 4.1) the oracle kernel and the scheduler are parameters of the reduction relation rather than components of the configuration, and the shared store, channel state, and logical clock are made explicit.

Configurations reduce in one of two modes: REC (live execution) or REP (crash recovery). The mode is a global parameter of the reduction relation, written $\langle \mathcal{A},\ \mu,\ \mathcal{J},\ \mathfrak{s} \rangle \rightsquigarrow_{\text{REC}} C'$ or $\rightsquigarrow_{\text{REP}}$.

3.3 Primitive 1: Agent

An *Agent* $A = \langle id, \Sigma, B \rangle$ is the unit of composition and failure containment. Σ is its private state (a partial function from keys to labeled values); B is its current behaviour.

Agents compose by parallel composition: $\mathcal{A} = \{A_1, \ldots, A_n\}$ is a pool of concurrently running agents. Delegation $\mathbf{delegate}(B_1, c)$ spawns a fresh agent with behaviour B_1, whose result is returned on channel c.

Definition 3.4 (Agent contract)**.** An *agent contract* is a proposition $\varphi(x)$ over result terms together with a probability bound $p \in [0, 1]$. We write $A \models_p \varphi$ to mean

$$\Pr[A \text{ terminates with output satisfying } \varphi] \geq p$$

(quantified over all schedulers).

3.4 Primitive 2: Oracle

The **Oracle** primitive is the sole source of probabilistic nondeterminism in the calculus.

Definition 3.5 (Oracle)**.** An *oracle* $\mathcal{O} = (\mathcal{E}, \{\mu_e\}_{e \in \mathcal{E}})$ consists of a set of prompts $\mathcal{E}$ and, for each prompt e, a probability measure μ_e on result terms (over the Borel σ-algebra on a suitable term space).

The syntactic form $x \leftarrow \mathbf{oracle}(e);\ B$ reduces in record mode by sampling $x \sim \mu_e$ and appending the pair $(e \mapsto x)$ to the journal. In replay mode it reads the next entry $(e \mapsto x)$ from the journal and proceeds deterministically. The formal reduction rules appear in Section 4.

Remark 3.6 (Generality)**.** The Oracle primitive is intentionally general: it models LLM inference calls, tool calls with stochastic outcomes, random seeds, and any other step whose result is drawn from a distribution. The concrete LLM interface is one instantiation; the calculus does not depend on any particular model.

Remark 3.7 (Abstract measure)**.** We do not require μ_e to be known or computable. Properties are stated either *distribution-agnostically* (for all μ) or *parametrically* (given assumptions on μ, e.g. a tail bound or a calibration condition). This keeps proofs tractable without depending on unrealistic knowledge of the LLM's true next-token distribution.

3.5 Primitive 3: Channel

Channels carry the *adversarial* (demonic) nondeterminism. They are π-calculus-style names with session-typed interaction disciplines.

Definition 3.8 (Channel and scheduler)**.** A *channel* c has an *interaction type* T specifying the sequence and types of messages it must carry. A *scheduler* $\mathfrak{s}$ is a function that, given the current configuration, selects which pending channel synchronization (from among all enabled actions) occurs next. The set of all schedulers is $\mathfrak{S}$.

Remark 3.9 (Two-layer nondeterminism discipline)**.** The design rule is: *all probabilistic nondeterminism flows through Oracle; all adversarial nondeterminism flows through Channel schedulers.* These two layers are never conflated. When stating probabilistic properties, we fix a scheduler $\mathfrak{s}$ and measure over Oracle draws; when proving worst-case properties, we quantify over $\mathfrak{s} \in \mathfrak{S}$. This separation is the standard Segala-Lynch discipline [34] applied to the LLM-agent setting.

Channel receives are journaled (alongside Oracle draws) so that replay is sound for externally-determined values regardless of their origin.

3.6 Primitive 4: Memory

The **Mem** primitive is a store with three dimensions: scope, retention, and IFC label.

Definition 3.10 (Memory)**.** A *memory store* Σ is a partial function $\mathcal{N} \to \mathcal{L} \times \mathcal{V}$ mapping keys to labeled values. Each cell (k, ℓ, v) has:

- a *scope* $s \in \{\mathsf{private}, \mathsf{group}, \mathsf{global}\}$ controlling which agents may access it,
- a *retention* $r \in \{\mathsf{turn}, \mathsf{session}, \mathsf{durable}\}$ controlling its lifetime, and
- an *IFC label* $\ell \in \mathcal{L}$ governing information-flow.

Each agent carries a *current label* that rises as it reads data. A read $\mathbf{read}(k^\ell)$ is permitted if ℓ is within the agent's clearance, and it raises the agent's current label to include ℓ. A write $\mathbf{write}(k^\ell, v)$ is permitted only if the agent's current label is below ℓ (no write-down).

Context is modelled as a bounded *sub-store* with turn or session retention. Context truncation, when the agent's working window exceeds its bound, is an explicit reduction rule, not a silent side effect. This is essential: silent truncation is a common source of information-flow violations in deployed LLM agents (Section 9.2 discusses the residual risks).

3.7 Primitive 5: Journal

The **Journal** primitive is the core of our durability result.

Definition 3.11 (Journal)**.** A *journal* $\mathcal{J}$ is a finite sequence of *events* $e_1, e_2, \ldots, e_n$, each recording an externally determined value: an oracle draw, a channel send or receive, a durable-effect intent or result, or a clock read (Definition 4.2). Each event carries a data-independent key identifying the step that produced it. The journal is *append-only*: events are only ever added.

Definition 3.12 (Checkpoint)**.** A *checkpoint* $\mathsf{Cursor} = |\mathcal{J}|$ is the length of the journal at a *durable* point (a point at which the journal has been persisted to stable storage). After a crash, the journal is truncated to the last checkpoint length Cursor, and replay begins from the configuration that produced $e_1, \ldots, e_{\mathsf{Cursor}}$.

The two reduction modes operate as follows:

- **Record mode (REC):** when an oracle draw $x \leftarrow \mathbf{oracle}(e)$ fires, the result $x \sim \mu_e$ is sampled and the event $(e \mapsto x, \mathsf{oracle})$ is appended to $\mathcal{J}$.
- **Replay mode (REP):** when an oracle draw $x \leftarrow \mathbf{oracle}(e)$ fires, the next event in $\mathcal{J}$ is consumed and x is set to its stored value deterministically.

The formal rules are in Section 4. Theorem T1 in Section 5 establishes that replay-mode execution is observationally equivalent to and equiprobable with the original record-mode execution.

3.8 Primitive 6: Supervisor

The **Sup** primitive handles failure propagation and recovery.

Definition 3.13 (Fault and decision)**.** A *fault* $\lightning$ is one of:

- OracleErr s — an oracle call raises an error,
- ToolErr s — an external tool call fails,
- Timeout — a deadline is exceeded,
- BudgetExhausted — a *budget* capability (step or cost limit) is consumed, or
- ContractViol φ — an agent contract is falsified.

A supervisor's *decision* d is one of: restart (replay from the journal, then resume), compensate($saga$) (run compensating actions from a saga list), escalate (re-raise to the parent), or stop (abandon the protected term).

Compensation is defined against the journal: an agent can only compensate effects that were journaled. This gives the *no-orphaned-effect* property: no durable side effect of a failed agent remains without either successful completion or compensation.

Remark 3.14 (Budget capability)**.** A *budget capability* $\mathsf{bud}(n)$ carried by an agent limits it to at most n steps (or n units of cost). When exhausted, a $\mathsf{BudgetExhausted}$ fault fires. This converts an unbounded loop, a common cause of runaway production agents, into a well-typed fault with a defined supervisor response.

3.9 Syntactic Sugar and Derived Forms

For readability we use the following common abbreviations:

$$
\begin{aligned}
B_1;\ B_2 &\triangleq B_1 \text{ followed by } B_2, \text{ where } B_1 \text{ binds no variable used in } B_2 \\
\mathbf{loop}(B) &\triangleq \mu X.\ B;\ X \quad \text{(used only under a budget capability)} \\
\mathbf{try}(B, \lightning \Rightarrow d) &\triangleq \mathbf{try}\ B\ \mathbf{handle}\ \{\lightning \mapsto d\}
\end{aligned}
$$

These are definitional conveniences; the primitive forms suffice for all formal arguments.

3.10 Summary

Table 2 summarizes the six primitives, their syntactic role, and their key formal property.

Table 2: The six Agentical primitives.

Primitive	Novel?	Main role	Key property
Agent	No	Composition unit	Failure containment; per-agent contracts
Oracle	Yes	Probabilistic LLM call	All quantitative guarantees flow through here
Chan	No	Communication	Session-typed progress; adversarial nondet. source
Mem	No	Labeled persistent state	IFC confidentiality + retention guarantees
Journal	Yes	Event history	Replay soundness under nondeterminism (T1)
Sup	No	Fault handling	No-orphaned-effect (Lemma 5.4)

4 Operational Semantics

4.1 Syntax

This section fixes the core syntax used in all formal results; the surface forms of Section 3 abbreviate it. Table 3 summarises the principal sorts and metavariables, and Appendix A collects the full notation.

Design invariants. The rules below are designed to satisfy four invariants, on which the proofs of Section 5 rely.

(I1) All probabilistic nondeterminism flows through the **Oracle**: only the oracle-draw rule has a non-Dirac distribution on its right-hand side.

(I2) All adversarial nondeterminism (interleaving, message ordering, the choice of which agent steps) is resolved by a scheduler; no probability is placed on it.

(I3) The oracle measure is abstract: no rule inspects the internal structure of μ_e.

(I4) Every externally determined value (oracle draws, channel receives, clock reads, durable-effect results) is journaled in the same step in which it is consumed, before it can influence a committed effect.

Table 3: Sorts and metavariables of the Agentical calculus.

Sort	Metavars	Meaning
Values	v, w	closed, irreducible results
Expressions	e	pure, deterministic computations
Terms (processes)	M, N	agent behaviour under reduction
Agents	A	$id[M]_\Sigma$
Systems	S	parallel pool of agents
Channel names	c, d	π-calculus names
Memory keys	k	cells in the labeled store
IFC labels	$\ell \in (\mathcal{L}, \sqsubseteq)$	lattice; $\mathsf{Low} \sqsubseteq \mathsf{High}$
Journal keys	$\mathsf{key} \in \mathcal{K}$	stable step identifiers
Agent identities	$id \in AID$	stable agent names

Values and expressions.

$$v, w \;::=\; () \mid b \mid \lambda x.\, M \mid \langle v_1, \ldots, v_n \rangle \mid \mathsf{inl}\; v \mid \mathsf{inr}\; v \mid c$$

Expressions e are *pure and deterministic*: the big-step judgment $e \Downarrow v$ is a (partial) deterministic function. All stochasticity is confined to the oracle term below (design invariant I1).

Terms.

$$\begin{aligned} M, N \;::=\;& \mathbf{return}\; v \mid \mathbf{let}\; x = e\; \mathbf{in}\; M \mid \mathbf{let}\; x \Leftarrow \mathbf{oracle}(e)\; \mathbf{in}\; M \\ \mid\;& M_1 \parallel M_2 \mid c\,!\,e\,.\,M \mid c\,?\,x\,.\,M \mid \nu c\,.\,M \\ \mid\;& \mathbf{read}[\ell, \sigma]\, k\; \mathbf{as}\; x\,.\,M \mid \mathbf{write}[\ell, \sigma, \rho]\, k := e\,.\,M \\ \mid\;& \mathbf{trunc}(\gamma)\,.\,M \mid \mathbf{do}[f]\, e\; \mathbf{as}\; x\,.\,M \\ \mid\;& \mathbf{spawn}\; id\langle e \rangle\; \mathbf{awaiting}\; c\,.\,M \mid \mathbf{fault}(\,\lightning\,)\,.\,M \mid \mathbf{try}\; M\; \mathbf{handle}\; h \mid \mu X\,.\,M \mid X \end{aligned}$$

Fault tags: $\lightning \;::=\; \mathsf{OracleErr}\; s \mid \mathsf{ToolErr}\; s \mid \mathsf{Timeout} \mid \mathsf{BudgetExhausted} \mid \mathsf{ContractViol}\; \varphi$. Handler clauses h map fault tags to decisions $D \in \{\mathsf{restart}, \mathsf{compensate}(saga), \mathsf{escalate}, \mathsf{stop}\}$.

4.2 Configurations

Definition 4.1 (System configuration)**.** A *configuration* is a five-tuple

$$C \;=\; \langle P;\, H;\, J;\, K;\, t \rangle$$

where:

- P is a finite multiset of *agents* $id[M]_\Sigma$ (the agent pool); each private state Σ includes the agent's *current label* $pc(\Sigma) \in \mathcal{L}$ (Section 4.5);
- $H : k \rightharpoonup (\ell, \sigma, \rho, v)$ is the *shared labeled store* (group/global cells);
- J is the *journal*: a finite sequence of events (see Definition 4.2);
- K maps channel names to queues of in-flight values plus the set of restricted names; each channel c carries a fixed IFC label ℓ_c;
- $t \in \mathbb{N}$ is a monotone *logical clock* (clock reads are journaled).

We write $C.P$, $C.J$, etc. for projections. A configuration is *closed* if every free channel name is in $\mathrm{dom}(K)$ and every memory key is defined in H or in the appropriate private store Σ.

Definition 4.2 (Journal events)**.** The journal alphabet is:

Event ::=	$\mathsf{OracleDraw}(k, v, prov)$	oracle draw at key k, outcome v, provenance
|	$\mathsf{ChannelRecv}(k, v)$	channel receive at key k, value v
|	$\mathsf{ChannelSend}(k, v)$	channel send at key k, value v
|	$\mathsf{Effect}(k, f, v, s)$	durable effect f at k, value v, status s
|	$\mathsf{Clock}(k, t)$	realized clock value

where $s \in \{\mathsf{intent}, \mathsf{committed}, \mathsf{compensating}\}$. Journal operations: $\mathsf{append}(J, e) = J \cdot e$ and $\mathsf{lookup}(J, k)$ returns the unique event with key k, or $\bot$.

Events carry a *journal key* that is the *cursor pair*

$$\mathsf{key}(C, id) \;=\; (id,\; cur_{id})$$

where $cur_{id} \in \mathbb{N}$ is the monotone per-agent counter of journaled steps taken by id so far. The key involves no hash and no program counter: the cursor is a simple count, strictly increasing per journaled step within one agent and distinct across agents by the id component. The key is *data-independent*: it does not depend on any realized random value, which is what makes replay sound (Lemma 5.1). Per-agent journal operations:

$$\mathsf{append}(J, id, e) = J \text{ with } e \text{ appended at } id\text{'s next cursor.}$$

$$\mathsf{replay_next}(J, id) = \begin{cases} \mathsf{Replay}(e) & id\text{'s cursor} < |J \restriction id| \\ \mathsf{Fresh} & \text{otherwise} \end{cases}$$

4.3 Structural Congruence

The structural congruence $\equiv$ is the least congruence satisfying:

$$S \parallel 0 \;\equiv\; S \qquad S_1 \parallel S_2 \;\equiv\; S_2 \parallel S_1 \qquad (S_1 \parallel S_2) \parallel S_3 \;\equiv\; S_1 \parallel (S_2 \parallel S_3)$$

$$\nu c \,.\, 0 \;\equiv\; 0 \qquad \nu c \,.\, \nu d \,.\, S \;\equiv\; \nu d \,.\, \nu c \,.\, S$$

$$\nu c \,.\, (S_1 \parallel S_2) \;\equiv\; (\nu c \,.\, S_1) \parallel S_2 \quad \text{if } c \notin \mathsf{fn}(S_2) \quad \text{(scope extrusion)}$$

$$\mu X \,.\, M \;\equiv\; M[\mu X \,.\, M / X] \quad \text{(recursion unfolding)}$$

Reduction is taken up to $\equiv$.

4.4 Observables and Traces

Fix a downward-closed set $Lo \subseteq \mathcal{L}$ of *observable labels* (the attacker's view for T2). The *observation* of C is:

$$\mathrm{obs}_{Lo}(C) \triangleq \big(\{(k,v) \in H : \mathsf{label}(k) \in Lo\},\ \text{low-visible channel buffers},\ \text{low-visible outputs}\big).$$

A *trace* is the sequence of transition labels produced by a finite reduction sequence together with the terminal observation. Two sequences are *observationally equivalent* ($\approx$) if they agree on the trace projection under obs (full labels for T1; *Lo*-labels for T2).

4.5 Reduction Rules

The reduction relation comes in two modes $m \in \{\textsc{rec}, \textsc{rep}\}$. A transition has the form

$$C \xrightarrow{a}_m \mathcal{D}$$

where a is a *transition label* and $\mathcal{D}$ is a sub-probability distribution over successor configurations. Transition labels: $a ::= \tau \mid o(k,\ell) \mid c\,!\,v \mid c\,?\,v \mid \iota(k,\ell) \mid \omega(k,\ell) \mid \mathsf{eff}(k,f) \mid \mathsf{spw}(id') \mid \lightning$.

Convention. Every rule except [O-REC] produces a Dirac distribution $\mathcal{D} = \delta_{C'}$ on a single successor C'; we write those rules in point-mass form as ordinary arrows. Only [O-REC] produces a non-Dirac $\mathcal{D}$ (design invariant I1).

We use an *evaluation context* E to fix the next redex within an agent's term (leftmost-outermost within each agent): $E ::= [\cdot] \mid \mathbf{let}\ x = E\ \mathbf{in}\ M \mid \mathbf{let}\ x \Leftarrow \mathbf{oracle}(E)\ \mathbf{in}\ M \mid \ldots$

Pure reduction.

$$\frac{e \Downarrow v}{\langle id[E[\mathbf{let}\ x{=}e\ \mathbf{in}\ M]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle \xrightarrow{\tau}_m \delta_{\langle id[E[M[v/x]]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle}} \ [\textsc{E-Let}]$$

Oracle draw, record mode ([O-REC]). This is the *only* rule with a non-Dirac successor distribution. The oracle term carries an explicit IFC output label ℓ, which must dominate the label of the prompt, so an oracle output is at least as secret as anything it was computed from.

$$\frac{e \Downarrow p \quad k = \mathsf{key}(C, id) \quad \ell \sqsupseteq pc(\Sigma) \quad \mu_p \text{ a probability measure on } Val}{\langle id[E[\mathbf{let}\ x \Leftarrow \mathbf{oracle}[\ell](e)\ \mathbf{in}\ M]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle \xrightarrow{o(k,\ell)}_{\textsc{rec}} \lambda(dv).\ \delta_{\langle id[E[M[v/x]]]_\Sigma \parallel P;\ H;\ J \cdot \mathsf{OracleDraw}(k,v,prov);\ K;\ t\rangle} \text{ weighted by } \mu_p(dv)} \ [\textsc{O-rec}]$$

Formally, for measurable $\Psi \subseteq \mathsf{Configs}$:

$$\mathcal{D}_{\textsc{rec}}(\Psi) \;=\; \int_{Val} \mathbf{1}[C'(v) \in \Psi]\ \mu_p(dv)$$

where $C'(v)$ is the successor with v substituted for x and journaled. After the draw the agent's current label becomes ℓ (elided from the display); since $\ell \sqsupseteq pc(\Sigma)$, an oracle output is at least as secret as everything the prompt could have been computed from. The sampled value is appended to the journal *in the same atomic step* as it is consumed (invariant I4).

Oracle draw, replay mode ([O-REP]). Replay is Dirac: the journal provides the outcome deterministically.

$$\frac{e \Downarrow p \quad k = \mathsf{key}(C, id) \quad \mathsf{replay_next}(J, id) = \mathsf{Replay}(\mathsf{OracleDraw}(k, v, prov))}{\langle id[E[\mathbf{let}\ x \Leftarrow \mathbf{oracle}[\ell](e)\ \mathbf{in}\ M]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle \xrightarrow{o(k,\ell)}_{\mathrm{REP}} \delta_{\langle id[E[M[v/x]]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle}} \text{[O-REP]}$$

Properties of [O-REP]: (a) the label $o(k, \ell)$ is *identical* to the record-mode label, so replay is invisible in the trace alphabet; (b) the transition is Dirac (probability 1); (c) the journal J is *unchanged* (read, not extended), so replaying a prefix is idempotent; (d) the key k is data-independent, so record and replay address the same journal entry (Lemma 5.1).

Oracle mode switch ([O-RESUME]). When $\mathsf{replay_next}(J, id) = \mathsf{Fresh}$ (the replay prefix for this agent is exhausted), the rule falls through to [O-REC]:

$$\frac{e \Downarrow p \quad k = \mathsf{key}(C, id) \quad \mathsf{replay_next}(J, id) = \mathsf{Fresh}}{C \xrightarrow{\cdots}_{\mathrm{REP}} \cdots \triangleq C \xrightarrow{\cdots}_{\mathrm{REC}} \cdots \quad \text{(fall through to [O-REC])}} \text{[O-RESUME]}$$

Channel, record mode. Sends are *journaled* in record mode. This makes the send-suppression decision during replay (below) locally computable at the sender, without querying another agent's journal.

$$\frac{e \Downarrow v \quad k_s = \mathsf{key}(C, id)}{\langle id[E[c!\,e\,.\,M]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle \xrightarrow{c!\,v}_{\mathrm{REC}} \delta_{\langle id[E[M]]_\Sigma \parallel P;\ H;\ J \cdot \mathsf{ChannelSend}(k_s, v);\ K[c \mapsto K(c) \cdot v];\ t\rangle}} \text{[SND-REC]}$$

$$\frac{K(c) = v \cdot q \quad k = \mathsf{key}(C, id)}{\langle id[E[c?\,x\,.\,M]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle \xrightarrow{c?\,v}_{\mathrm{REC}} \delta_{\langle id[E[M[v/x]]]_\Sigma \parallel P;\ H;\ J \cdot \mathsf{ChannelRecv}(k, v);\ K[c \mapsto q];\ t\rangle}} \text{[RCV-REC]}$$

Receives are journaled because the value arrives from outside the receiving agent (invariant I4); sends are journaled so that replay can decide locally whether a send must be re-delivered.

Channel, replay mode. Replay reads the received value from the journal. Send replay uses a *two-variant* dispatch against the consistent cut U from which recovery resumes (assumption A-CUT, Section 5); the predicate $\mathsf{matchedRecv}(J, id, c, v, U)$ reports whether the receive matching this send lies below U.

$$\frac{k = \mathsf{key}(C, id) \quad \mathsf{replay_next}(J, id) = \mathsf{Replay}(\mathsf{ChannelRecv}(k, v))}{\langle id[E[c?\,x\,.\,M]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle \xrightarrow{c?\,v}_{\mathrm{REP}} \delta_{\langle id[E[M[v/x]]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle}} \text{[RCV-REP]}$$

$$\frac{\mathsf{matchedRecv}(J, id, c, v, U) = \mathsf{BelowCut}}{\langle id[E[c!\,e\,.\,M]]_\Sigma \parallel P; \ldots\rangle \xrightarrow{\tau}_{\mathrm{REP}} \delta_{\langle id[E[M]]_\Sigma \parallel P; \ldots\rangle}} \text{[SND-REP-SUPPRESS]}$$

$$\frac{e \Downarrow v \quad \mathsf{matchedRecv}(J, id, c, v, U) = \mathsf{LiveOrAboveCut}}{\langle id[E[c!\,e\,.\,M]]_\Sigma \parallel P;\ H;\ J;\ K;\ t\rangle \xrightarrow{c!\,v}_{\mathrm{REP}} \delta_{\langle id[E[M]]_\Sigma \parallel P;\ H;\ J;\ K[c \mapsto K(c) \cdot v];\ t\rangle}} \text{[SND-REP-DELIVER]}$$

Remark 4.3 (Two-variant send suppression)**.** [Snd-rep-suppress] fires when the matching receiver's ChannelRecv entry is below the consistent cut U — i.e., the message was durably delivered before the crash. [Snd-rep-deliver] fires when the receiver is live or above the cut, the message must be re-delivered to prevent deadlock. The predicate matchedRecv is locally computable because sends are journaled ([Snd-rec]) and the cut is a checkable function of per-agent checkpoint cursors. A blanket rule that suppressed every replayed send would deadlock a live peer still waiting for the message; with re-delivery, that peer finds the message in K.

Memory, labeled, scope/retention-checked (both modes). Memory is internally determined and hence *not* journaled; it is reconstructed by replay.

$$\frac{H(k) = (\ell', \sigma, \rho, v) \quad \ell' \sqsubseteq \ell \sqsubseteq \mathsf{clearance}(id) \quad \mathsf{readable}(\sigma, id) \quad \mathsf{live}(\rho, t)}{\langle id[E[\mathbf{read}[\ell, \sigma]\, k\ \mathbf{as}\ x\,.\,M]]_\Sigma \parallel P;\, H;\, J;\, \ldots;\, t\rangle \xrightarrow{\iota(k,\ell')}_m \delta_{\langle id[E[M[v/x]]]_{\Sigma[pc \mapsto pc(\Sigma)\sqcup\ell']} \parallel P;\, H;\, J;\, \ldots;\, t\rangle}} \quad [\textsc{Mem-Rd}]$$

$$\frac{e \Downarrow v \quad \mathsf{writable}(\sigma, id) \quad pc(\Sigma) \sqsubseteq \ell}{\langle id[E[\mathbf{write}[\ell, \sigma, \rho]\, k{:=}e\,.\,M]]_\Sigma \parallel P;\, H;\, J;\, \ldots;\, t\rangle \xrightarrow{\omega(k,\ell)}_m \delta_{\langle id[E[M]]_\Sigma \parallel P;\, H[k \mapsto (\ell,\sigma,\rho,v)];\, J;\, \ldots;\, t\rangle}} \quad [\textsc{Mem-Wr}]$$

Floating labels. Information flow is controlled by a coarse-grained *floating label* discipline in the style of LIO [36]. Each agent carries a current label $pc(\Sigma)$, initially its starting label. Reading a cell raises it to $pc(\Sigma) \sqcup \ell'$ ([Mem-Rd]), and no agent may read above its clearance. A write is permitted only to a cell whose label dominates the writer's current label, $pc(\Sigma) \sqsubseteq \ell$ (no write-down, [Mem-Wr]). Channels follow the same discipline, with side conditions elided from the rule displays: a send on c requires $pc(\Sigma) \sqsubseteq \ell_c$; because a receive consumes a message (a destructive read), it also requires $pc(\Sigma) \sqsubseteq \ell_c$, and it raises the receiver's label to ℓ_c. A spawned agent inherits its parent's current label, and an escalation delivered to a parent is treated as a receive. Since current labels only rise, an agent that has observed High data can never again write, send, or draw at a Low label; this is what rules out implicit flows without tracking labels on individual values. Context truncation is explicit:

$$\frac{}{\langle id[E[\mathbf{trunc}(\gamma)\,.\,M]]_\Sigma \parallel P; \ldots\rangle \xrightarrow{\tau}_m \delta_{\langle id[E[M]]_{\Sigma\restriction\gamma} \parallel P; \ldots\rangle}} \quad [\textsc{Trunc}]$$

where $\Sigma \restriction \gamma$ retains only the cells the budget γ specifies. Modeling truncation explicitly makes dropping context a visible step of the semantics rather than a silent side effect outside the model.

Durable effect, three-phase write-ahead discipline. A durable effect is split into three separate small steps: [Eff-intent] journals the intent *before* performing f; [Eff-perform] executes f in the world; [Eff-commit] journals the committed result. This exposes the crash windows precisely and supports per-class recovery (Lemma 5.4).

$$\frac{e \Downarrow v \quad k = \mathsf{key}(C, id)}{\langle id[E[\mathbf{do}[f]\, e\ \mathbf{as}\ x\,.\,M]]_\Sigma \parallel P;\, H;\, J;\, \ldots;\, \rangle \xrightarrow{\tau}_{\mathrm{REC}} \delta_{\langle id[E[\mathbf{do}[f]\, v\ \mathbf{as}\ x\,.\,M]]_\Sigma \parallel P;\, H;\, J \cdot \mathsf{Effect}(k,f,v,\mathsf{intent});\, \ldots;\, \rangle}} \quad [\textsc{Eff-intent}]$$

$$\frac{f(v) \downarrow_W (w, W') \quad k = \mathsf{key}(C, id) \quad \mathsf{Effect}(k, f, v, \mathsf{intent}) \in J}{\langle id[E[\mathbf{do}[f]\, v\ \mathbf{as}\ x\,.\,M]]_\Sigma \parallel P;\, H;\, J;\, \ldots;\, \rangle \xrightarrow{\mathsf{eff}(k,f)}_{\mathrm{REC}} \delta_{\langle id[E[\mathbf{do}[f]_{\mathsf{done}(w)}\ \mathbf{as}\ x\,.\,M]]_\Sigma \parallel P;\, H;\, J;\, \ldots;\, \rangle}} \quad [\textsc{Eff-perform}]$$

$$\frac{k = \mathsf{key}(C, id)}{\langle id[E[\mathbf{do}[f]_{\mathsf{done}(w)}\ \mathbf{as}\ x\,.\,M]]_\Sigma \parallel P;\ H;\ J;\ \ldots;\ \rangle \xrightarrow{\tau}_{\mathrm{REC}} \delta_{\langle id[E[M[w/x]]]_\Sigma \| P;\ H;\ J \cdot \mathsf{Effect}(k,f,w,\mathsf{committed});\ldots;\rangle}} \quad [\textsc{Eff-commit}]$$

$$\frac{k = \mathsf{key}(C, id) \quad \mathsf{replay_next}(J, id) = \mathsf{Replay}(\mathsf{Effect}(k, f, w, \mathsf{committed}))}{\langle id[E[\mathbf{do}[f]\, e\ \mathbf{as}\ x\,.\,M]]_\Sigma \parallel P;\ H;\ J;\ \ldots;\ \rangle \xrightarrow{\mathsf{eff}(k,f)}_{\mathrm{REP}} \delta_{\langle id[E[M[w/x]]]_\Sigma \| P;\ H;\ J;\ldots;\rangle}} \quad [\textsc{Eff-rep}]$$

Effect f is *not* re-performed during replay. If a crash leaves $\mathsf{Effect}(k, f, v, \mathsf{intent})$ without a matching $\mathsf{Effect}(k, f, w, \mathsf{committed})$, the recovery rule fires:

$$\frac{\mathsf{Effect}(k, f, v, \mathsf{intent}) \in J \quad \mathsf{Effect}(k, f, -, \mathsf{committed}) \notin J \quad \mathsf{cls}(f) = cls}{\langle \text{resume at } k \rangle \ \Rightarrow\ \text{dispatch on } cls \in \{I, C, A, X\} \text{ per Lemma 5.4}} \quad [\textsc{Eff-recover}]$$

Fault and supervisor.

$$\frac{}{\langle id[E[\mathbf{fault}(\lightning)\,.\,M]]_\Sigma \parallel P; \ldots\rangle \xrightarrow{\lightning}_m \delta_{\langle id[\langle \text{raised } \lightning\rangle]_\Sigma \| P;\ldots\rangle}} \quad [\textsc{Flt}]$$

$$\frac{\langle \text{raised } \lightning\rangle \text{ inside } \mathbf{try}\ N\ \mathbf{handle}\ h \quad h(\lightning) = \mathsf{restart}}{\langle id[\mathbf{try}\ N\ \mathbf{handle}\ h]_\Sigma \parallel P;\ H;\ J;\ \ldots;\ \rangle \xrightarrow{\tau}_{\mathrm{REC}} \delta_{\langle id[\mathsf{replay}(N,\, J \upharpoonright id)]_\Sigma \| P;\ H;\ J;\ldots;\rangle}} \quad [\textsc{Sup-Restart}]$$

$\mathsf{replay}(N, J \upharpoonright id)$ re-enters the protected term in REP mode against the journal projection for id. This is the operational definition of "restart from journal" and is precisely what T1 (Section 5.2) certifies sound. Compensation is analogous; escalation (not shown) re-raises $\lightning$ to the enclosing handler, and stop discards the protected term:

$$\frac{h(\lightning) = \mathsf{compensate}(saga) \quad saga = [comp_1, \ldots, comp_m]}{\langle id[\mathbf{try}\ N\ \mathbf{handle}\ h]_\Sigma \parallel P;\ldots\rangle \xrightarrow{\tau}_m \delta_{\langle id[comp_m;\cdots;comp_1;\langle \text{handled}\rangle]_\Sigma \| P;\ldots\rangle}} \quad [\textsc{Sup-Comp}]$$

Compensations run in *reverse order* against the committed $\mathsf{Effect}(-,-,-,\mathsf{committed})$ entries in $J \upharpoonright id$.

4.6 Probabilistic Transition System

Definition 4.4 (Probabilistic automaton)**.** The configurations of the Agentical calculus, together with the transition relation of Section 4.5, form a *probabilistic automaton* (PA) in the sense of Segala and Lynch [34]. At each configuration C, the set of *enabled transitions* $\mathsf{enabled}(C)$ is the collection of pairs $(a, \mathcal{D})$ arising from the rules. By invariant I1, the only non-Dirac $\mathcal{D}$ arises from [O-REC], where $\mathcal{D} = \mu_p(\cdot)$. All other $\mathcal{D}$ are Dirac.

Schedulers. A *scheduler* $\mathfrak{s}$ resolves adversarial nondeterminism:

$$\mathfrak{s} : \big(\text{finite history } C_0 \xrightarrow{a_1} \cdots \xrightarrow{a_n} C_n\big) \ \longrightarrow\ (a, \mathcal{D}) \in \mathsf{enabled}(C_n) \cup \{\bot\}.$$

We quantify over *all* schedulers $\mathfrak{s} \in \mathfrak{S}$ unless stated otherwise. For T2, schedulers are further required to be *Lo-observable over the filtered history* filt_{Lo} (High steps appear as τ) *and deterministic given their filtered view* — assumption (A-SCHED). Without it, a scheduler could leak High control flow by branching on the labels of High steps (see Section 5.3).

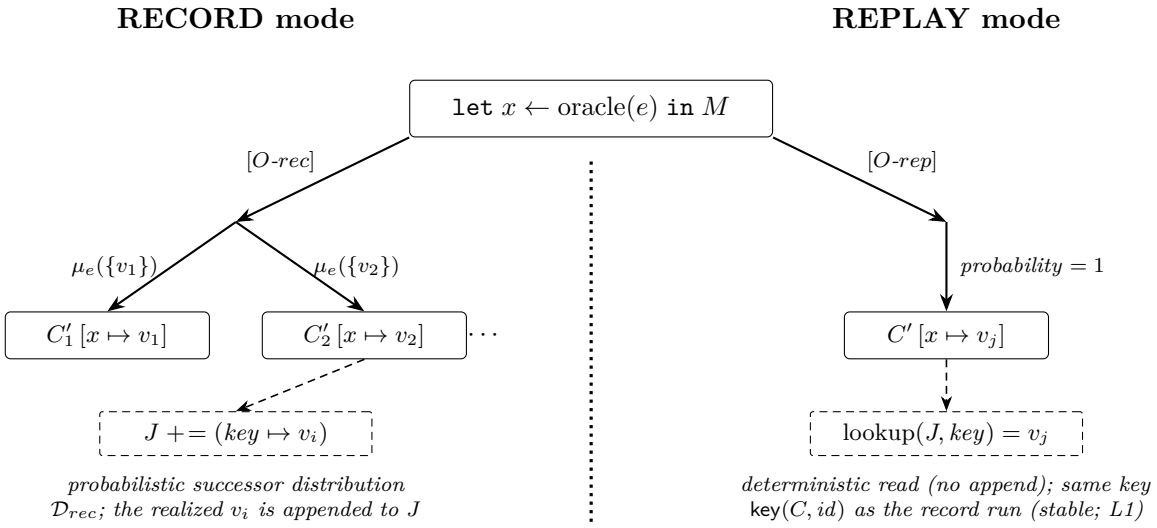


Figure 1: The two-mode oracle reduction. In *record* mode (left) the draw $x \leftarrow \text{oracle}[\ell](e)$ samples outcome $x \sim \mu_e$ and appends $(key \mapsto x)$ to the journal J. In *replay* mode (right) the same term reduces deterministically (Dirac, probability 1) by reading the stored outcome; no new randomness is consumed. The key (id, cur) is data-independent (Lemma 5.1), so record and replay land on the same journal entry.

Ionescu–Tulcea trace measure. Fix an initial configuration C_0 and a scheduler $\mathfrak{s}$. Resolving the adversarial choice at every step with $\mathfrak{s}$ collapses the PA to a (history-dependent) Markov chain. By the Ionescu–Tulcea theorem, the per-step kernels $\{\mathcal{D}\}$ together with the initial point mass δ_{C_0} extend to a unique probability measure

$$\mathbb{P}_{\mathfrak{s},C_0} \quad \text{on finite and infinite execution paths.}$$

All probability statements in this paper are statements about $\mathbb{P}_{\mathfrak{s},C_0}$ for a quantified scheduler $\mathfrak{s}$. Concretely, a safety property "φ holds with probability $\geq p$" means $\forall \mathfrak{s}.\ \mathbb{P}_{\mathfrak{s},C_0}[\varphi] \geq p$.

Two-layer factorization. Because invariant I1 confines all measures μ to [O-REC] and invariant I2 confines all adversarial choice to scheduling and channels,

$$\mathbb{P}_{\mathfrak{s},C_0} \;=\; \big(\text{adversary } \mathfrak{s} \text{ picks the step-sequence skeleton}\big) \;\circ\; \Big(\prod_{\text{oracle draws}} \mu_{p_i}\Big).$$

This factorization is what enables the T1 coupling: replay *freezes* the oracle randomness (by reading J) while re-using the same scheduler skeleton, so record and replay coincide in every observable component.

4.7 Well-Labelled Journals

Definition 4.5 (Well-labelled journal)**.** A journal J is *well-labelled* with respect to a closed initial configuration C_0 and a finite reduction sequence R if:

(i) $J = C_n.J$ is the journal produced by R;

(ii) every oracle draw in R corresponds to exactly one $\mathsf{OracleDraw}(k, v, prov) \in J$, in order, with matching key $k = \mathsf{key}(C_i, id)$ at the step i where agent id performed the draw; and

(iii) every channel receive in R corresponds to exactly one $\mathsf{ChannelRecv}(k, v) \in J$, in order, with matching key.

We write $\mathsf{WL}(J, R)$ to mean J is well-labelled with respect to R.

Well-labeledness is the precondition for Theorem T1 (Section 5.2). The cursor-pair key function guarantees it holds for every record-mode run (Lemma 5.1).

Rule summary. Table 4 indexes the full rule set.

Table 4: Reduction rules of the Agentical calculus.

Rule	Mode	Det/Prob	Journals?	Primitive
[E-Let]	both	det	—	local
[O-rec]	record	**prob** (μ_p)	append OracleDraw	**Oracle**
[O-rep]	replay	det (Dirac)	reads journal	**Oracle**
[O-resume]	replay→record	—	—	**Oracle/Journal**
[Snd-rec]/[Rcv-rec]	record	det/adv	send appends ChannelSend; recv appends ChannelRecv	**Chan**
[Rcv-rep]/[Snd-rep-suppress]/[Snd-rep-deliver]	replay	det	reads journal; suppress below-cut or re-deliver	**Chan**
[Mem-Rd]/[Mem-Wr]/[Trunc]	both	det	—	**Mem**
[Eff-intent]/[Eff-perform]/[Eff-commit]	record	det/effect	append intent, perform f, append committed	**Journal**
[Eff-rep]	replay	det	reads committed entry	**Journal**
[Eff-recover]	resume	det	class dispatch on intent without committed	**Journal/Sup**
[Flt]/[Sup-*]	both	det	reads $J \upharpoonright id$	**Sup**

5 Metatheory

This section states and proves the three main theorems of the Agentical calculus. The following standing assumptions are in force throughout.

(A1) Journal keys are the cursor pairs (id, cur) of Section 4.2.
(A2) Replay re-enters the same term (up to $\equiv$) as the recorded run.
(A3) Durable effects follow the three-phase write-ahead rules [Eff-intent], [Eff-perform], [Eff-commit].
(A4) Pure evaluation $e \Downarrow v$ is a deterministic partial function, and effects use the separate judgment $f(v) \downarrow_W (w, W')$.
(A5) Each oracle measure μ_p is a probability measure on values; nothing else is assumed about it unless a theorem says so (invariant I3).
(A6) The journal persisted to stable storage is neither corrupted nor reordered; the only truncation is a crash that keeps a prefix.
(A-CUT) Recovery resumes from a *consistent cut* U of the per-agent checkpoint cursors [7]: if a receive lies below U, so does the matching send.

Oracle outputs of LLMs are finite token sequences, so in practice each μ_p is a discrete measure on a countable set; we use this only to speak of the probability of an individual finite run.

5.1 Supporting Lemmas

Lemma 5.1 (Key/cursor determinism and freshness, L1)**.** *Under* (A1)*: for a fixed initial configuration C_0, a fixed scheduler skeleton, and a fixed realized-outcome assignment, the key (id, cur) assigned to the n-th journaled step of agent id is (i)* the same *in a record run and the corresponding replay run, and (ii)* distinct *from the key of every other journaled step.*

Proof. The key is (id, cur) where *cur* is the count of journaled steps *id* has taken. (i) Replay re-enters the same term (A2), journals nothing on [O-REP]/[RCV-REP]/[EFF-REP] (Lemma 5.3), and advances the counter identically per journaled step, so the n-th journaled step gets cursor n in both runs, with no dependence on any realized value. (ii) Within one agent *cur* strictly increases per journaled step; across agents the *id* component differs. Injectivity is structural; no hash assumption is needed. □

Lemma 5.2 (Determinism of non-oracle steps, L2)**.** *Every reduction rule except* [O-REC] *produces a Dirac distribution $\delta_{C'}$ on a unique successor C'. The label a and successor C' are functions of C and the rule's side conditions only (no measure μ).*

Proof. By inspection of Section 4.5. [E-LET]/[E-RET]/[TRUNC] use $e \Downarrow v$, deterministic by (A4). [SND-*]/[RCV-*] are deterministic given K, the journal, and (for replayed sends) the consistent-cut predicate matchedRecv, a deterministic function of the cut. [MEM-*] are deterministic given H. The three-phase effect rules use $e \Downarrow v$ and the world judgment $f(v) \downarrow_W (w, W')$: the world nondeterminism is externally determined and journaled (invariant I4), so each rule produces a single successor. [FLT]/[SUP-*] are deterministic given handler h. None produces a non-Dirac distribution; the only rule with a measure on its right-hand side is [O-REC]. □

Lemma 5.3 (Replay is locally Dirac and journal-non-extending, L3)**.** *For any configuration C whose next step fires a replay rule* ([O-REP], [RCV-REP], [SND-REP-SUPPRESS], [SND-REP-DELIVER], [EFF-REP])*: the transition is $C \xrightarrow{a}_{\text{REP}} \delta_{C'}$ (probability 1), the label a equals the label the corresponding record rule produced modulo the projection π (which erases only below-cut suppressed sends), and $C'.J = C.J$ (the journal is read, not extended).*

Proof. Direct from the replay rules in Section 4.5: each has a Dirac right-hand side; none appends to J. Labels: [O-REP] $\Rightarrow o(k, \ell)$; [RCV-REP] $\Rightarrow c\,?\,v$; [EFF-REP] $\Rightarrow \mathsf{eff}(k, f)$ — identical to their record counterparts. [SND-REP-DELIVER] $\Rightarrow c\,!\,v$, identical to [SND-REC]. [SND-REP-SUPPRESS] $\Rightarrow \tau$; this is the only label discrepancy, and it is exactly the case π erases. □

Lemma 5.4 (No orphaned durable effect, L4)**.** *Under* (A3)*, in every reachable configuration:*

(a) every effect performed in the world has its intent entry $\mathsf{Effect}(k, f, v, \mathsf{intent})$ *earlier in J; and*
(b) if a continuation consumed an effect result w, then $\mathsf{Effect}(k, f, w, \mathsf{committed})$ *occurs earlier in J.*

Consequently, after a crash the only effects whose status is unknown are those with an intent entry but no committed entry, and [EFF-RECOVER] *resolves each of them by its effect class:*

- ***Class I (idempotent):*** *re-perform with the journaled argument v and commit.*
- ***Class A (atomic):*** *query the external transaction's durable outcome; commit or abort accordingly.*
- ***Class C (compensable):*** *a probe* $\mathsf{probe}(f, v)$ *reports* $\mathsf{committed}(w)$ *or* $\mathsf{not\text{-}started}$*; the probe result is journaled (invariant I4), and recovery commits or re-performs. This relies on assumption* (A-EFF-C)*: the probe is sound.*
- ***Class X (non-recoverable):*** *raise* ContractViol *and escalate. Assumption* (A-EFF-X) *states that no Class-X effect is reachable in a recoverable scope.*

Compensation ([SUP-COMP]*) journals each compensating entry before running the next compensation, so a crash part-way through a saga [13] does not undo any step twice.*

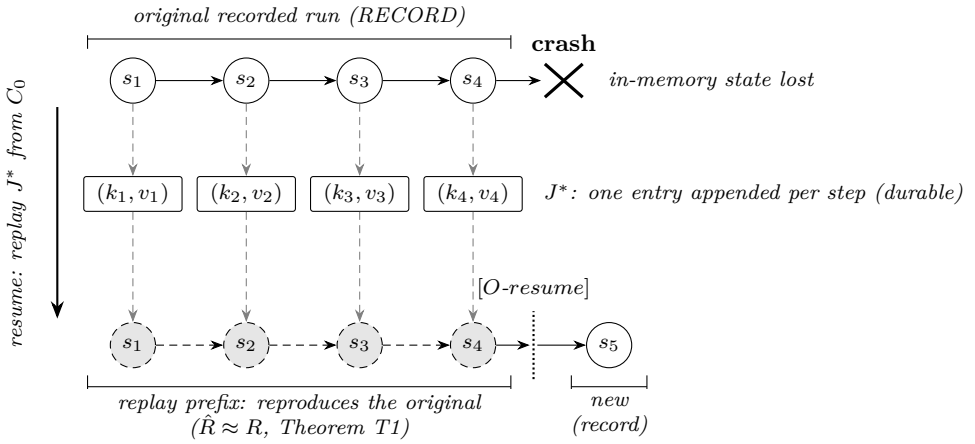


Figure 2: Crash-recovery timeline (Corollary C1). Steps $s_1 \dots s_4$ execute in *record* mode; each oracle/receive outcome is appended to the durable journal J^*. A crash at step s_5 loses all in-memory state. On resume the system re-enters *replay* mode from C_0, feeding J^*: each step fires the deterministic rule [O-REP] and recovers the same outcome with probability 1 (Theorem T1, part (ii)). When J^* is exhausted, rule [O-RESUME] transitions back to *record* mode and new steps are appended. The resumed continuation's law equals the original law conditioned on the replayed prefix (Markov property of $\mathbb{P}_{\mathfrak{s},C_0}$).

Proof. (a) and (b) hold by induction on the length of the reduction sequence, by inspection of the three effect rules: [EFF-PERFORM] has $\mathsf{Effect}(k, f, v, \mathsf{intent}) \in J$ as a premise, and [EFF-COMMIT] is the only rule that substitutes an effect result into the continuation, appending the committed entry in the same step. No rule removes journal entries (A6). For the recovery cases: re-performing a Class-I effect leaves the world in the same state whether or not the first attempt took place, by idempotence; for Class A the external system's atomicity guarantees that its recorded outcome is the only possible one; for Class C the journaled probe result is correct by (A-EFF-C); Class X is excluded by (A-EFF-X). In each case the effect ends with exactly one committed or compensated entry. □

5.2 T1: Replay Soundness Under Nondeterminism

Theorem T1 is the central technical result. It says that the Journal primitive recovers durable replay without assuming determinism: a replayed run is both observationally equivalent to and equiprobable with the original.

Theorem 5.5 (Replay soundness, T1)**.** *Let $C_0 = \langle P_0; H_0; \varepsilon; K_0; t_0 \rangle$ be a closed initial configuration with empty journal, and fix any scheduler $\mathfrak{s}$. Run the system in* record *mode under $\mathfrak{s}$; suppose it reaches with positive probability a configuration C_n after a finite reduction sequence $R = (C_0 \xrightarrow{a_1}_{\mathrm{REC}} \cdots \xrightarrow{a_n}_{\mathrm{REC}} C_n)$ realizing journal $J^* = C_n.J$.*

Let the replay run *start from C_0 (same term, (A2)) in replay mode against the frozen J^*, driven by the coupled scheduler $\hat{\mathfrak{s}}$ (defined in §5.2.1). Then:*

(i) ***(Trace equivalence)*** *The replay run produces a sequence $\hat{R} = (C_0 \xrightarrow{\hat{a}_1}_{\mathrm{REP}} \cdots \xrightarrow{\hat{a}_n}_{\mathrm{REP}} C'_n)$ with the same label sequence modulo π: $\pi(a_1 \cdots a_n) = \pi(\hat{a}_1 \cdots \hat{a}_n)$, where π erases only below-cut suppressed sends, and $\mathrm{obs}_{\mathcal{L}}(C'_n) = \mathrm{obs}_{\mathcal{L}}(C_n)$ (the observation over all labels). In short, $\hat{R} \approx R$.*

(ii) ***(Determinism of replay)*** *$\hat{R}$ occurs with probability 1:* $\mathbb{P}^{\mathrm{REP}}_{\hat{\mathfrak{s}},C_0}[tr = \hat{a}_1 \cdots \hat{a}_n] = 1$.

(iii) ***(Equiprobability)*** *The record run's probability of this trace-and-journal is:*

$$\mathbb{P}^{\mathrm{REC}}_{\mathfrak{s},C_0}[tr = a_1 \cdots a_n,\ J = J^*] \ = \prod_{\text{oracle draws } i \text{ in } J^*} \mu_{p_i}(\{v_i\}).$$

Record and replay are a coupling of the same underlying randomness; any trace-measurable event has equal probability under the record law conditioned on J^* and under replay.

Hypotheses (scope): *(A1)–(A6) hold. For durable effects, additionally: each reachable effect is Class I or A (unconditionally recoverable), or Class C under (A-EFF-C); no Class-X effect is reachable (A-EFF-X). Without the effect hypotheses, parts (i)–(iii) hold for the oracle-and-channel fragment (no* ***do****[f]) unconditionally.*

5.2.1 The coupling and the $\hat{\mathfrak{s}} \to \mathfrak{s}$ handover

The proof constructs a *coupling* between the record run and the replay run: a single probability space $\Omega = Val^{\mathbb{N}}$ on which both runs are defined and provably agree step-for-step.

Sample once. Let $\omega = (x_1, x_2, \ldots) \sim \prod_i \mu_{p_i}$ be the sequence of oracle outcomes the record run draws (in the order the scheduler fires oracle steps). The record reduction R is a deterministic function of ω and the scheduler $\mathfrak{s}$ (by Lemma 5.2, every non-oracle step is determined; the only randomness consumed is ω). Write $R = \mathrm{Rec}(\mathfrak{s}, \omega)$ and $J^* = \mathrm{Journal}(\mathrm{Rec}(\mathfrak{s}, \omega))$.

Coupled scheduler. Define $\hat{\mathfrak{s}}$ for replay to make the same choices $\mathfrak{s}$ made in R: at history $\hat{h}$, $\hat{\mathfrak{s}}(\hat{h})$ fires the transition with the same label that $\mathfrak{s}$ fired at the corresponding position of R. This is well-defined by induction: at each step the two runs are at configurations with the same enabled label.

Handover. When $\mathsf{replay_next}(J^*, id) = \mathsf{Fresh}$, rule [O-RESUME] fires the record rule and from that step on $\hat{\mathfrak{s}}$ *equals* $\mathfrak{s}$ (nothing left in J^* to replay for id). The handover is a well-defined function of position, not an ad-hoc switch.

Both runs now live on the same ω: the record run *draws* ω; the replay run *reads* ω out of J^*.

Proof of T1. We prove parts (i)–(ii) by induction on step index $i = 0, \ldots, n$, establishing the invariant:

($\mathbf{INV}_i$) After i steps, C_i (record) and C'_i (replay) satisfy: (a) same agent terms up to $\equiv$, same shared and private memory, and same clock; and the same channel contents, except that the record run may additionally hold in-flight messages whose matching receive occurs later in J^* (in the replay run those sends are suppressed and the receives are served from the journal); (b) journal prefixes agree on the first i journaled events; (c) label sequences agree modulo π; (d) replay step i had probability 1.

Base case $i = 0$: $C_0 = C'_0$ (A2). (INV_0) holds.

Inductive step: Assume (INV_i). Scheduler $\hat{\mathfrak{s}}$ fires the transition with the same label a_{i+1} that $\mathfrak{s}$ fired in R. By (INV_i(a)) the configurations are equal up to $\equiv$, so the same rule is enabled on both with the same side conditions. Cases:

- *Non-oracle, non-replay-special* ([E-LET], [MEM-*], etc.): by Lemma 5.2, the same deterministic rule on C'_i produces C'_{i+1} with $C_{i+1} \equiv C'_{i+1}$. Label equal, probability 1. (INV_{i+1}) holds.
- *Oracle draw* (record [O-REC], replay [O-REP]): Record drew $x_j = \omega_j$, appended $\mathsf{OracleDraw}(k, x_j, prov)$ at cursor (id, cur). Replay: by (INV_i(a)) the same $e \Downarrow p$; by Lemma 5.1 the same cursor; by (INV_i(b)) the journal entry is exactly $\mathsf{OracleDraw}(k, x_j, prov)$, so replay_next returns it and [O-REP] substitutes the same x_j. $C_{i+1} \equiv C'_{i+1}$. Label $o(k, \ell)$ on both (Lemma 5.3). Replay Dirac (d). (INV_{i+1}) holds.
- *Channel receive* (record [RCV-REC], replay [RCV-REP]): Symmetric with $\mathsf{ChannelRecv}(v)$. Replay reads the same v from J^* at the same cursor and ignores live K. This is exactly why receives are journaled (I4). (INV_{i+1}) holds.
- *Channel send during replay* (record [SND-REC], replay [SND-REP-SUPPRESS] or [SND-REP-DELIVER]): By (INV_i(b)) the send event $\mathsf{ChannelSend}(v)$ is in J^* at id's cursor. The consistent cut U (A-CUT) decides:
  - If the matching receiver's $\mathsf{ChannelRecv}(v)$ is below the cut: [SND-REP-SUPPRESS] fires (label τ); π erases it. No message is lost: the recipient already has it below the cut.
  - If the matching receiver is live or above the cut: [SND-REP-DELIVER] fires (label $c\,!\,v$, re-delivers into K). No deadlock: the live peer finds the message it awaits.

  In both sub-cases (INV_{i+1}(a)) holds: a suppressed message is exactly one whose receive is in J^*, which is the exception allowed in (INV(a)), and A-CUT guarantees that the suppress/deliver choice mirrors whether delivery already happened. At $i = n$ every message whose receive is in J^* has been received in the record run as well, so the channel contents of C_n and C'_n coincide.
- *Durable effect* (record three-step, replay [EFF-REP]): Under the effect hypotheses (Class I/A, or C under A-EFF-C; no X): replay reads the committed w from J^* (Lemma 5.4) and does not re-perform f. Successors equal, label $\mathsf{eff}(k, f)$ on both, replay Dirac. (INV_{i+1}) holds.

This completes the induction. (INV_n(c)) $\Rightarrow$ part (i); (INV_n(a)) $\Rightarrow$ equal observations; (INV_n(d) at every step) $\Rightarrow$ part (ii).

Part (iii) (equiprobability). On $(\Omega, \prod_i \mu_{p_i})$: $\mathrm{Rec}(\mathfrak{s}, \omega)$ is a deterministic function of ω by Lemma 5.2. The probability of realizing this trace with this journal is the measure of the set of ω whose oracle reads equal the values in J^*, which by product-measure independence is $\prod_{i:\mathrm{oracle}} \mu_{p_i}(\{x_i\})$. The replay run is, on the same ω, the deterministic function $\mathrm{Rep}(\hat{\mathfrak{s}}, J^*(\omega))$; by parts (i)–(ii) it equals $\mathrm{Rec}(\mathfrak{s}, \omega)$ (modulo π) for every ω in the positive-probability event that produced J^*. Two random variables equal on a coupling have equal law; therefore for any trace-measurable Φ, $\mathbb{P}^{\mathrm{REC}}_{\mathfrak{s}, C_0}[\Phi \mid J = J^*] = \mathbb{P}^{\mathrm{REP}}_{\hat{\mathfrak{s}}, C_0}[\Phi]$. □

5.2.2 Corollaries

Corollary 5.6 (Crash-resume soundness, C1)**.** *Suppose a crash truncates the run at a consistent cut U* (A-CUT)*: each agent id has a checkpoint cursor cur_{id}, and U is consistent. Resume = every agent replays its prefix $J^* \restriction id$ up to cur_{id} (deterministically, probability 1, by T1 part (ii)), with cross-scope sends handled by* [SND-REP-SUPPRESS]/[SND-REP-DELIVER] *against U, then* [O-RESUME] *to record mode for the remainder. The resumed run's continuation law equals the original run's law conditioned on the replayed prefix.*

Proof. Replaying each agent's prefix is an instance of T1 restricted to the prefix, so it reproduces the prefix with probability 1. Because U is consistent, every receive below U has its send below U, so [SND-REP-SUPPRESS] drops exactly the sends already delivered and [SND-REP-DELIVER] re-delivers the rest: no message is lost or duplicated. After the prefix, [O-RESUME] continues in record mode from a configuration equal to the one the original run reached, and the trace measure has the Markov property, so the continuation's law is the original law conditioned on that prefix. Without a consistent cut this argument fails: a receive below the cut whose send lies above it would be replayed from the journal while the send is re-executed and delivered again. □

Corollary 5.7 (Restart soundness, C2)**.** [SUP-RESTART]*'s* $\mathsf{replay}(N, J \restriction id)$ *yields, by T1, a run observationally equivalent to and equiprobable with the faulted child's recorded prefix, provided the restart scope satisfies A-CUT.*

Corollary 5.8 (Idempotent re-replay, C3)**.** *Replaying twice yields the same trace (T1 parts (i)–(ii) are deterministic in J^*). Compensation re-replay is also idempotent because* [SUP-COMP] *journals* $\mathsf{Effect}(-,-,-,\mathsf{compensating})$ *entries (Lemma 5.4).*

5.3 T2: Multi-Agent Probabilistic Noninterference

T2 is an information-flow result for concurrently running agents: under the floating-label discipline of Section 4.5, an observer who sees only Low data learns nothing about High data, even statistically, through memory, channels, oracle calls, or scheduling. LLMbda [12] proves a probabilistic noninterference theorem for a single agent's program; T2 addresses what concurrency adds. As is well known for multi-threaded programs [32, 40], a scheduler that can observe how long secret computations take turns timing into a channel, so T2 restricts the scheduler.

Setup. We consider the fragment without durable effects and clock reads: both expose the external world or timing, and the calculus does not label them. Fix a set $Lo \subsetneq \mathcal{L}$ of observable labels that is downward closed and closed under joins (the attacker's view). A transition is *Low-visible* if the stepping agent's current label before the step is in Lo and the data the step reads, writes, sends, receives, or draws has a label in Lo (the cell label, the channel label ℓ_c, or the oracle output label); every other transition is *High*. The *Low trace* tr_{Lo} of a path is its subsequence of Low-visible labels: High steps are deleted, not replaced by τ. Two configurations are *low-equivalent*, $C =_{Lo} C'$, if (i) they have the same agent identities, each with the same current label on both sides; (ii) every agent whose current label is in Lo has the same term and the same Lo-labelled private cells on both sides; and (iii) they agree on all Lo-labelled shared cells and on the contents of all Lo-labelled channels. Agents whose current label is not in Lo, High cells, and High channels may differ arbitrarily.

Assumptions for T2.

- **(WL) Floating labels:** the label side conditions of Section 4.5 hold (no read above clearance, no write-down, no send or receive below the current label, oracle output label above the current label).
- **(A-ORC-NI) Prompt-determined oracle:** the measure of an oracle call depends only on its prompt, with no state shared between calls. In the calculus this holds by construction (I3); in a deployment it is an architectural requirement (Section 9.2).

- **(A-SCHED) Low-observing scheduler:** the scheduler is deterministic and, at each step, fires the next transition of the highest-priority enabled agent under a priority order that is a function of the Low trace so far. It cannot observe High steps, not even their number.
- **(A-TERM) High phases terminate:** with probability 1, every maximal run of consecutive High steps is finite.

Theorem 5.9 (Probabilistic noninterference, T2)**.** *Assume* (WL)*,* (A-ORC-NI)*,* (A-SCHED)*, and* (A-TERM)*. For all closed* $C_0 =_{Lo} C_0'$ *and every scheduler* $\mathfrak{s}$ *satisfying* (A-SCHED)*, the Low traces have the same distribution:*

$$(tr_{Lo})_* \, \mathbb{P}^{\mathrm{REC}}_{\mathfrak{s},C_0} \;=\; (tr_{Lo})_* \, \mathbb{P}^{\mathrm{REC}}_{\mathfrak{s},C_0'}.$$

The guarantee covers the order of Low events, not only their multiset, because the scheduler cannot see High steps. Without (A-TERM) a High computation that diverges can stall the scheduler and withhold all later Low events; this is the termination channel that termination-insensitive guarantees conventionally leave open.

Proof sketch. We build a coupling of the two runs that shares the randomness of Low oracle draws and uses independent randomness for High draws, and show that the Low traces agree almost surely.

1. *High steps preserve low-equivalence.* If $C =_{Lo} C'$ and C takes a High step to D, then $D =_{Lo} C'$. A High step is taken either by an agent whose current label is not in Lo, or by a Low agent touching High data. In the first case, (WL) forbids writes, sends, and receives below the agent's label, so no Low cell or Low channel changes, and the agent's own entry is unconstrained by $=_{Lo}$. In the second case the step reads High data or draws at a High label (raising the agent's label out of Lo, after which its term is unconstrained), or writes a High cell or sends on a High channel (changing only High state). A spawned agent inherits its parent's label, so a High agent spawns only High agents.

2. *Low steps are matched.* If $C =_{Lo} C'$ and the same agent, with label in Lo, takes a Low-visible step on both sides, it executes the same rule on the same term against the same Low data, so it produces the same label and preserves $=_{Lo}$. For a Low oracle draw the prompt was computed only from Low data, so it is identical on both sides and, by (A-ORC-NI), drawn from the same measure; the coupling uses the same value.

3. *The next Low step is the same on both sides.* Fix low-equivalent configurations with the same Low history, so the scheduler uses the same priority order on both sides. The scheduler repeatedly fires the highest-priority enabled agent. Whether a Low agent's next Low-visible step is enabled depends only on its term and on Low cells and Low channels, so it is the same on both sides and, by step 1, unaffected by High steps. Steps of Low agents that keep their label in Lo but are High (writing a High cell, sending on a High channel) are deterministic and enabled on both sides alike; steps that raise a Low agent's label (reading High data, drawing at a High label, receiving on a High channel) remove that agent from all later Low activity, on whichever side they happen. High agents may be enabled on one side and not the other, but they only take High steps, and by (A-TERM) every run of High steps is finite almost surely. Hence, almost surely, both sides next fire a Low-visible step of the same agent: the highest-priority agent among those whose next Low-visible step is enabled. If there is no such agent, neither side produces another Low event.

By induction on the number of Low steps, using steps 1–3, the Low traces of the coupled runs agree almost surely. Two random variables that agree almost surely on a coupling have the same law, which gives the theorem. □

Scope of T2. The argument above is a proof sketch and is not mechanized. Two hypotheses carry semantic weight beyond the rules: (A-ORC-NI), which a deployment must ensure by not sharing model state (such as a cache or a system prompt) between labels, and (A-SCHED), which excludes schedulers that react to the progress of secret computations. Durable effects and clock reads are outside the fragment.

5.4 T3: Compositional Probability Bounds

T3 states how per-agent probability contracts combine into a global bound. The bounds themselves are elementary facts of probability theory; their role here is to show that the calculus supports this kind of reasoning, which a model that treats LLM outputs as arbitrary cannot express.

Setup. Each agent A_i of a system $S = A_1 \parallel \cdots \parallel A_N$ has a *local probabilistic contract* $\langle pre_i, \varphi_i, p_i \rangle$: started in any state satisfying pre_i, under any scheduler, its output satisfies φ_i with probability at least p_i. The agents communicate over channels whose protocol guarantees that each pre_i holds whenever A_i starts, and a global property Φ satisfies $\bigwedge_i \varphi_i \Rightarrow \Phi$. Then $\mathbb{P}_{\mathfrak{s},}[\varphi_i] \geq p_i$ for every i and every scheduler $\mathfrak{s}$.

Theorem 5.10 (Compositional probability bounds, T3)**.** *(a)* ***(Union bound; arbitrary dependence.)*** *For every scheduler* $\mathfrak{s}$*,*

$$\mathbb{P}_{\mathfrak{s},}[\Phi] \;\geq\; \max\Big(0,\; 1 - \sum_{i=1}^{N}(1 - p_i)\Big).$$

This needs no independence. It coincides with the Fréchet lower bound $\max(0, \sum_i p_i - (N-1))$*, which is attained by some joint distribution of the events* φ_i*, so no better bound holds under arbitrary dependence.*

(b) ***(Product bound; requires A-IND.)*** *If additionally* (A-IND) *holds (the contract events are mutually independent; in particular, the agents draw from oracle instances that share no state), then for every scheduler* $\mathfrak{s}$*,*

$$\mathbb{P}_{\mathfrak{s},}[\Phi] \;\geq\; \prod_{i=1}^{N} p_i.$$

(c) ***(Expected cost.)*** *For an additive cost* $cost = \sum_i cost_i$*: if* $\mathbb{E}_{\mathfrak{s}}[cost_i] \leq k_i$ *for each* i*, then* $\mathbb{E}_{\mathfrak{s}}[cost] \leq \sum_i k_i$*, with no independence needed.*

Proof. Fix $\mathfrak{s}$. (a) $\mathbb{P}_{,}[\neg\Phi] \leq \mathbb{P}_{,}[\bigvee_i \neg\varphi_i] \leq \sum_i \mathbb{P}_{,}[\neg\varphi_i] \leq \sum_i (1 - p_i)$ by Boole's inequality, and a probability is non-negative. The bound is the Fréchet lower bound for the conjunction of events with marginals $\geq p_i$, which is sharp. (b) Under (A-IND), $\mathbb{P}_{,}[\bigwedge_i \varphi_i] = \prod_i \mathbb{P}_{,}[\varphi_i] \geq \prod_i p_i$. A-IND must exclude shared oracle state: agents that call one LLM service can fail in a correlated way (for example under rate limits), which breaks the product bound but not (a). (c) Linearity of expectation; a budget capability (BudgetExhausted) bounds each $cost_i$, so the expectations are finite. □

5.5 Assumptions Ledger

Table 5 summarizes the lemmas and assumptions and their status.

Table 5: Lemmas and assumptions, with their status.

ID	Claim	Status	Used by
L1	Key (id, cur) stable and collision-free	proved; mechanized (single agent)	T1, L3
L2	Only [O-REC] is probabilistic	proved	T1
L3	Replay is Dirac and does not extend the journal	proved	T1
L4	No orphaned effect (Classes I, A; C under A-EFF-C)	proved	T1 with effects
A-CUT	Resume from a consistent cut	assumption (checkable)	C1, C2
A-EFF-C	Class-C probe is sound	assumption	L4, T1 with effects
A-EFF-X	No Class-X effect reachable	assumption	L4, T1 with effects
A-ORC-NI	Oracle measure depends only on the prompt	assumption (architectural)	T2
A-SCHED	Scheduler observes only the Low trace	assumption	T2
A-TERM	High phases terminate almost surely	assumption	T2
A-IND	Independent contract events and oracle instances	assumption	T3(b)
A2	Same term replayed	assumption	T1
A6	Journal not corrupted or reordered	assumption	T1

6 Applications

This section applies the calculus to four common multi-agent patterns. Each case study gives the calculus term (sometimes abbreviated), identifies the failure modes the model handles, and states which theorems and lemmas provide the guarantee. The recurring point is that the Journal, the write-ahead discipline, and the rules [Eff-intent], [Eff-perform], [Eff-commit], and [Eff-recover] turn crash recovery, usually a matter of engineering practice, into a procedure derived from the semantics.

6.1 Tool Use with Retries and Crash Recovery

Scenario. An ActionAgent calls an unreliable external tool (a durable Effect, e.g., posting a calendar event, executing a payment request, writing to a database). The tool call is not idempotent in general, unreliable, and observable externally. The agent also makes Oracle draws to decide the tool argument and to post-process the result. The process can be killed at any point in the sequence:

(A) Before any journal entry (pre-draw).
(B) After OracleDraw$(k_1, arg, prov)$ but before `Effect`(..., intent).
(C) After `Effect`(..., intent) but before `Effect`(..., committed), the hard case.

Calculus term.

$$\mathsf{ActionAgent}(id_A, goal, c_{\text{out}}) \triangleq id_A\Big[\ \textbf{try}$$
$$\quad \textbf{let } arg \Leftarrow \mathsf{oracle}(prompt_plan(goal)) \textbf{ in} \quad [\text{O-rec}]$$
$$\quad \textbf{let } result \Leftarrow \textbf{do}[calendar_post]\ arg \textbf{ as } r\ . \quad [\text{Eff-intent}]/[\text{Eff-perform}]/[\text{Eff-commit}]$$
$$\quad \textbf{let } reply \Leftarrow \mathsf{oracle}(prompt_interpret(result)) \textbf{ in}$$
$$\quad c_{\text{out}}!reply.\ \textbf{return}\ ()$$
$$\textbf{handle}\ \{\mathsf{ToolErr}\ s \mapsto compensate([comp]);\ \mathsf{BudgetExhausted} \mapsto compensate([comp]);\ \mathsf{Timeout} \mapsto restart\}\ \Big]_{\Sigma}$$

Recovery by crash window.

Window A (empty journal). replay_next(J, id_A) = Fresh; rule [O-resume] fires; the oracle draw is re-sampled. No external effect occurred; T1 does not apply (no journal prefix).

Window B (only OracleDraw in journal). [O-rep] replays *arg*; at the effect step the journal is exhausted, so execution falls through to record mode and the tool call runs once. Safe restart: oracle result frozen, tool call executed exactly once.

Window C (only `Effect(...,` intent) in journal). [Eff-rep] requires a committed entry and cannot fire. **Lemma 5.4 applies:** the Supervisor detects the orphan via `dangling_intents`, then runs compensation or an idempotent re-perform depending on the application-level assumption. The calculus does not prescribe which, it guarantees that *one of them runs*.

Formal decision procedure. Given $J\restriction_{id}$ after a crash:

- $\mathsf{OracleDraw}(k, v, _)$ present $\Rightarrow$ replay v via T1; do *not* re-sample.
- $\mathsf{Effect}(k, f, v, \mathsf{committed})$ present $\Rightarrow$ replay v via [Eff-rep]; do *not* re-run f.
- $\mathsf{Effect}(k, f, arg, \mathsf{intent})$ with no committed $\Rightarrow$ Lemma 5.4: compensation or idempotent re-run.
- Key absent entirely $\Rightarrow$ [O-resume]: fresh record from here.

Every crash point falls into one of these cases, so the procedure is derived from the journal and the reduction rules rather than from convention. Section 7.2 exercises it, for a single agent, at every journal position.

6.2 Map-Reduce Summarization

Scenario. N Summarizer workers each summarize one document chunk via an Oracle call and send the result over a ν-restricted private channel c_i. A Reducer collects all N summaries and fuses them with a final Oracle call.

System term (abbreviated).

$$\begin{aligned}&\mathsf{MapReduceSystem} \triangleq \nu c_1.\cdots\nu c_N.\nu c_{\text{out}}.\\ &\quad \mathsf{Coordinator} \parallel \mathsf{Summarizer}(id_1, doc_1, c_1) \parallel \cdots \parallel \mathsf{Reducer}(id_R, \{c_i\}, c_{\text{out}})\end{aligned}$$

The restriction νc_i ensures $c_i \notin \mathsf{fn}(\mathsf{Summarizer}_j)$ for $j \neq i$, so no summarizer can read another's channel.

T1, Replay-soundness. Each $\mathsf{Summarizer}_i$'s Oracle draw is recorded as $\mathsf{OracleDraw}(k_i, x_i, prov)$. If $\mathsf{Summarizer}_i$ is restarted, [O-rep] fires deterministically with x_i; the resumed run produces the same summary without a new LLM call. If the Reducer crashes after receiving $k < N$ summaries, it replays the first k ChannelRecv entries from the journal in $O(k)$ steps, then falls through to record mode and receives the remaining summaries live.

T3, Compositional bounds. If each $\mathsf{Summarizer}_i$ satisfies a local quality contract $\mathbb{P}[\varphi_i] \geq p_i$ ("output is a valid summary of doc_i"), then by Theorem 5.10:

$$\Big(\forall i.\, \mathbb{P}[\varphi_i] \geq p_i\Big) \implies \mathbb{P}\Big[\bigwedge_i \varphi_i\Big] \geq 1 - \sum_i (1 - p_i) \quad \text{(union bound, no independence assumption)}$$

If additionally A-IND holds, Theorem 5.10(b) gives the stronger product bound $\prod_i p_i$. Disjoint channels do not by themselves give A-IND: summarizers that call one shared model service can fail in a correlated way.

Key insight. Standard distributed systems achieve durability by making workers idempotent and relying on retries. When LLM outputs are stochastic, retrying re-samples and produces a different summary, making the partial results already sent inconsistent. The Journal + T1 combination freezes the *realized value* from the journal, not a fresh draw. Idempotence alone does not achieve this, because re-running a stochastic step is not idempotent.

6.3 Debate with Confidential Sources

Scenario. A Proposer and a Critic debate a question over Low-labelled channels, and a Judge publishes a Low-labelled verdict. Some of the material the agents could consult is confidential: a store of High-labelled documents, such as customer records. The security property is that the public transcript and the verdict reveal nothing about the confidential documents.

Label setup. We use the two-point lattice $\mathsf{Low} \sqsubseteq \mathsf{High}$. The debate channels and the verdict channel are labelled Low, and every confidential cell is labelled High. All three agents start at current label Low.

T2, Probabilistic noninterference. Let C_0 and C_0' be two instances of the system that differ only in the contents of the confidential store, so $C_0 =_{Lo} C_0'$. For every scheduler satisfying A-SCHED, and assuming A-ORC-NI and A-TERM, Theorem 5.9 gives

$$(tr_{Lo})_* \, \mathbb{P}^{\mathrm{REC}}_{\mathfrak{s},C_0} \;=\; (tr_{Lo})_* \, \mathbb{P}^{\mathrm{REC}}_{\mathfrak{s},C_0'} :$$

the distribution of public arguments and verdicts is the same whatever the confidential documents contain.

Why the construction supports T2.

1. An agent that reads a confidential cell has its current label raised to High by [Mem-Rd], and from then on the floating-label side conditions forbid it to send on the Low debate channels or to write Low cells.
2. An oracle call made after such a read must have an output label at least High (side condition of [O-rec]), so a model response that may reflect confidential content is itself High.
3. Every message the Judge receives was therefore produced by an agent whose current label was Low, from Low data only.

Consequence. The discipline is strict: an agent that consults the confidential store can no longer take part in the public debate. A design that wants confidential documents to inform the public outcome needs deliberate declassification, which the calculus does not provide (Section 9.2). What T2 guarantees is that no structural path, whether through memory, channels, oracle prompts, or scheduling, carries confidential content into the public transcript; whether a deployed model leaks across labels through shared state is the subject of assumption A-ORC-NI.

6.4 Hierarchical Delegation with Supervision

Scenario. A Planner decomposes a task via an Oracle draw and spawns K SubAgents. Each SubAgent calls the LLM, calls an external tool ($\mathbf{do}[f]$ durable effect), and returns its result on a private channel. The Planner collects the K results and synthesizes with a final Oracle call. The supervision tree mirrors the agent tree.

Fault coverage. The handler clause set

$$\{\mathsf{OracleErr} \mapsto \mathit{compensate},\ \mathsf{ToolErr} \mapsto \mathit{compensate},\\ \mathsf{BudgetExhausted} \mapsto \mathit{compensate},\ \mathsf{Timeout} \mapsto \mathit{compensate}\}$$

covers all fault tags reachable from a SubAgent's body. An algebraic fault type makes uncovered cases detectable: the programmer's intent is explicit in the term; no silent propagation.

Lemma L4, No-orphaned-effect. Each $\mathsf{SubAgent}_k$ makes a $\mathbf{do}[db_write]$ call. A crash between the intent and committed journal entries leaves an orphan. L4 (Lemma 5.4) guarantees:

- The orphan is *detected* (the journal makes it observable).
- The handler *fires* (the Supervisor decision always runs).
- The handler has *access to the journal* ($J \restriction_{id_k}$ is passed to the compensation, so the handler knows exactly what was done).

What L4 requires: the compensation *comp_db_write* is a semantic inverse of *db_write* (saga discipline), a per-application correctness assumption. What L4 *proves*: an interrupted effect is never silently left unresolved.

T1, Planner restart. The Planner's `handle {`$\lightning \mapsto$ *restart*`}` invokes [Sup-Restart]: replay from $J \restriction_{id_P}$. T1 guarantees the replay is observationally equivalent to the pre-crash configuration: the decomposition Oracle draw is replayed (same sub-task partition); the first k ChannelRecv entries are replayed from the journal (same sub-results, without re-blocking on live channels); the first missing receive is the handoff point where [O-resume] fires.

Key insight. Popular agent frameworks give, to our knowledge, no formal account of what happens when a sub-agent fails after it has already had side effects. The Journal + Supervisor combination gives a precise answer for every failure mode: (i) if the effect was committed — replay reuses it, no re-execution; (ii) if only intent is present, recovery dispatches on the effect class (L4); (iii) if no effect entry exists, restart is a no-op on external state. This is the saga pattern [13] placed inside a probabilistic calculus, where the steps between compensations may themselves be stochastic oracle draws.

7 Evaluation

We evaluate the Rust prototype (Section 9.1) on three questions: does replay reproduce a recorded run (E1), does recovery from a crash at any point preserve what was journaled and complete the run (E2), and what does journaling cost (E3)? These experiments test that the single-agent implementation follows the record/replay semantics; they do not test the multi-agent results (C1 for concurrent agents, T2, T3), which the prototype does not yet implement, and they use a seeded stand-in for the LLM.

Setup. The agent is a small loop with two branches. It draws `classify` from {`tool` : 5/6, `answer` : 1/6}. On `tool` it draws `tool-arg` from {`arg-A` : 3/5, `arg-B` : 2/5} and performs a durable effect `commit(arg)` under the three-phase write-ahead discipline; on `answer` it draws `answer`, whose distribution has a single outcome. A recorded run therefore has two or four journal entries. The oracle is `SeededOracle`, a categorical sampler driven by SplitMix64, and the journal is `InMemoryJournal`. All seeds come from a fixed SplitMix64 stream, so every experiment is reproducible. Measurements were taken on an Apple M5 Pro with 48 GB of memory, macOS 27.0.1, and rustc 1.93.0, in release builds. Each experiment is an integration test in the accompanying repository (`tests/eval_e1.rs`, `eval_e2.rs`, `eval_e3.rs`) run with `cargo test -release -test <name> - -nocapture`.

7.1 E1: Replay Determinism

For each of $N = 1{,}000$ trials we record a run with one seed, then replay the complete journal with a fresh oracle seeded independently. We compare the traces (each label carries its journal key and the realized value) and the final outputs. T1 parts (i)–(ii) predict identical traces in every trial.

Result. All 1,000 replays reproduced the recorded trace and output exactly. Every trial made at least one oracle draw, 810 took the branch with the durable effect, and the replay seed differed from the record seed in every trial, so agreement comes from the journal rather than from the oracle. Because replay never consults the live oracle, this outcome is expected by construction; the experiment checks that the implementation conforms to [O-REP] and [EFF-REP].

7.2 E2: Crash Recovery at Every Journal Position

For each of 1,000 recorded runs, and for every crash point k from 0 to the journal length, we keep only the first k journal entries, then resume in replay mode with a fresh oracle whose seed differs from the recording seed. If the crash point separates an effect's intent from its committed record, the resumed run stops at the dangling intent; we then apply Class-I recovery (re-perform with the journaled argument and commit) and resume again. For each resume we check that (a) the run completes with an output and leaves no dangling intent; (b) the first k journal entries are unchanged and every trace label whose entry survived is reproduced; (c) every oracle value lies in the support of its distribution and journal cursors stay dense; and (d) an effect whose committed record survived is not performed again.

Result. The sweep made 4,664 resumes. All 4,664 completed and reproduced their surviving prefix, 832 of them crashed inside an effect's write-ahead window and were recovered, and no surviving committed effect was performed again.

Conditional law. Corollary C1 says that the resumed continuation follows the original law conditioned on the replayed prefix. We check this on the prefix `classify = tool`: across 20,000 resumes with fresh seeds, the re-sampled `tool-arg` was `arg-A` with frequency 0.5989, against $3/5 = 0.6$ (standard error ≈ 0.0035).

Compensation. After a crash inside the write-ahead window followed by a compensating recovery (Class C), replaying the journal twice stops both times at the compensated effect, produces no output, performs no effect, and appends nothing, as Corollary C3 requires.

A defect found by E2. An earlier version of the driver stayed in replay mode after [O-RESUME] fell through to recording, so the next step re-read the entry just appended as if it were history and could produce out-of-support values. The original test suite missed this because its only crash test took the branch whose distribution has a single outcome; the sweep above fails on that version and passes on the corrected one.

7.3 E3: Journaling Overhead

Append latency. We append $n \in \{10^3, 10^4, 10^5, 10^6\}$ oracle-draw events to an in-memory journal, after one warm-up pass, and report the mean time per append. Over six runs, every measurement fell between 88 and 145 ns per append, with no growth in n, consistent with amortized

Table 6: Summary of the evaluation.

ID	Question	Result relates to	Outcome
E1	Replay determinism	T1 (i)–(ii), single agent	1,000/1,000 identical replays
E2	Crash at every position	C1, C3, L4 (single agent)	4,664/4,664 resumes correct; no duplicate effects; conditional law matches
E3	Journaling overhead	implementation	88–145 ns per append, flat in n
—	Multi-agent recovery, T2, T3	C1, T2, T3	not evaluated (no multi-agent runtime)

constant-time append; the journal holds exactly one entry per step. These are single timed passes per size, not a statistical benchmark.

Replay versus re-execution. To model a networked LLM call we inject a latency of 1 ms into every oracle draw. A recorded run then takes about 2.51 ms, while replaying it takes 0.5–0.8 μs, a speedup of about 3,200× to 4,900× across six runs. The speedup is dominated by the injected latency: replay performs no oracle calls, so the time saved per replayed draw is the latency of the call itself.

Not measured. We did not measure the cost of journaling relative to a journal-free execution mode, which the prototype lacks, or the cost of persisting the journal (for example, `fsync` on every append), since no persistent backend exists yet.

7.4 Threats to Validity

Internal. The seeded oracle stands in for an LLM. The experiments show that the implementation follows the record/replay semantics, not how real models behave. Crashes are simulated by truncating an in-memory journal; torn writes, `fsync` behavior, and storage failures are not exercised.

Construct. Trace comparison in E1 and E2 includes realized values, which is stronger than the label equality in T1, so it can only make the checks stricter. E2 checks the conditional law on one prefix and one prompt, not in general.

External. All measurements come from one machine and a single in-memory backend, and the workload is a small agent; results for persistent backends, larger agents, or many concurrent agents may differ.

8 Related Work

This section situates Agentical within four bodies of work: formal models of LLM agents, probabilistic semantics, information-flow control for concurrent programs, and rollback recovery and durable execution. Table 1 in Section 2 summarizes the comparison with the closest systems: LLMbda has a mechanized probabilistic semantics for a single agent; MSC coordination is multi-agent, mechanized, and implemented but treats model outputs as arbitrary; and Temporal provides durability but requires deterministic workflow code.

8.1 Formal Models of LLM Agents

LLMbda and FIDES. LLMbda [12] is an untyped call-by-value λ-calculus whose constructs include prompt-response conversations that can be forked and cleared, code generation, and dynamic information-flow control in which every value carries a label. Its first version followed FIDES [9] in assuming that the underlying model is deterministic and named a probabilistic semantics, in the style of Borgström et al. [5], as the way to lift that assumption. The current version does so: its central result is a termination-insensitive probabilistic noninterference theorem for the whole calculus, its interpreter is itself the agent harness that calls the model, and all proofs are in Lean. FIDES is a planner-based system that enforces information-flow policies around an LLM; its formal guarantees are stated for a deterministic model function. Agentical is complementary to LLMbda: LLMbda studies one agent's program in depth, with value-level labels, whereas Agentical studies many concurrently scheduled agents, durable replay, and crash recovery, with a coarser, agent-level floating label. T2 can be read as asking what LLMbda-style probabilistic noninterference requires once agents run concurrently under a scheduler.

Other calculi and languages for agents. λ_A [26] extends the simply typed λ-calculus with oracle calls, bounded fixpoints, and probabilistic choice, and proves type safety and termination in Coq. PPDL [27] treats LLM-based flows as probabilistic programs whose semantics is a weighted distribution over traces. Zhou, D'Antoni, and Polikarpova [44] have agents generate well-typed Haskell programs whose information flow is controlled with LIO, and prove a soundness theorem with the model treated as a deterministic oracle. All three are single-agent and sequential, and none addresses replay or crash recovery.

MSC coordination calculus. Bollig, Függer, and Nowak [4] specify multi-agent workflows as message sequence charts in a domain-specific language that separates message-passing structure from LLM calls, tool calls, and human control points, project global specifications to deadlock-free local programs, and machine-check all results in Lean 4. Their implementation, ZipperGen, runs workflows through a persistent store and can replay the committed history after a restart, although this durability is not part of the formal model. The calculus establishes coordination properties independently of LLM nondeterminism, so model outputs are treated as arbitrary. Agentical takes the opposite approach to the model, treating each output as a draw from a measure, which is what allows probabilistic statements such as T3; MSC's guarantees, in exchange, hold for any model. MSC is not a session-type system; it contrasts its projection with multiparty session types.

Process-calculus semantics for tool protocols. Schlapbach [33] gives a process-calculus semantics for agentic tool protocols such as the Model Context Protocol. The second version of that preprint was withdrawn because of a logical flaw in its Theorem 21, and a revised version has since been posted. It does not address probabilistic behaviour or durability.

Semantic Fusion. Zaichyk [42] studies decentralized coordination in which agents operate on scoped views of a shared, ontology-structured memory and propose updates that are validated locally. Its probabilistic results include a bisimulation between an agent's local execution and its projection of the global execution when update proposals are generated probabilistically, and an analysis of validators that accept invalid proposals with small probability. The probability distributions there range over structured update proposals rather than over model outputs in general, and the framework does not address journaling, replay, or crash recovery; recovery is by re-synchronization from shared state.

Empirical work. Cemri et al. [6] build a taxonomy of multi-agent failures from more than 1,600 annotated traces of seven frameworks. Khatchadourian [20] proposes a protocol for replaying tool-using financial agents and comparing runs at different levels (final decisions, tool sets, ordered tool sequences); it is an empirical harness, not a semantics. He and Yu [14] argue, in a short position paper without formal results, that distributed systems built from agents must tolerate divergent execution traces that reach equivalent outcomes.

8.2 Probabilistic Semantics

Probabilistic program semantics goes back at least to Kozen [21]. Probabilistic automata [34] combine nondeterministic and probabilistic choice and resolve the former with schedulers; we adopt this two-layer view. Larsen and Skou [24] introduced probabilistic bisimulation. The probabilistic asynchronous π-calculus of Herescu and Palamidessi [15] likewise separates internal probabilistic choice from scheduler-resolved nondeterminism, and shows that probabilistic choice makes some problems solvable, such as symmetric leader election, that the asynchronous π-calculus cannot solve. Hillston's PEPA [16] is a stochastic process algebra whose activities carry exponential rates; our **Oracle** instead draws from an arbitrary measure over results. Borgström et al. [5] give a probabilistic call-by-value λ-calculus with distribution-based big-step and small-step semantics, which they prove equivalent, and a sampling-based semantics that agrees with them when integrated over traces. The proof of T1 is a coupling argument; probabilistic couplings are a standard tool for relational reasoning about probabilistic programs [2]. For LLM agents, Bensalem et al. [3] argue for chaining probabilistic assume-guarantee contracts to obtain system-level bounds and leave the multi-agent case open, and Solko-Breslin et al. [35] bound the probability of policy violations without independence assumptions; T3 states the corresponding elementary bounds inside the calculus.

8.3 Information-Flow Control for Concurrent Programs

Volpano and Smith [40] and Sabelfeld and Sands [32] showed that noninterference for multi-threaded programs must account for the scheduler: if scheduling can depend on how long secret computations take, timing becomes an information channel. T2 inherits this lesson through assumption (A-SCHED). Our floating-label discipline follows coarse-grained dynamic IFC systems such as LIO [36], in which a computation's current label rises as it observes data. For LLM agents, APPA [22] uses LIO-style floating labels with recoverable child branches and proves isolation and recovery properties for a single agent, and Adams [1] obtains noninterference for language models by secure multi-execution, running one instance per security level. Peters-Gill et al. [30] show empirically that single-agent defences against prompt injection do not compose across agent hierarchies, which motivates a noninterference result stated for concurrently scheduled agents, as T2 is.

8.4 Rollback Recovery and Durable Execution

Message logging. Logging every nondeterministic event so that a process can be replayed after a failure is a classical technique. Strom and Yemini's optimistic recovery [37] and the message-logging protocols surveyed by Elnozahy et al. [11] assume that processes are *piecewise deterministic*: execution is deterministic between nondeterministic events, which are logged and replayed. Consistent global states and cuts are due to Chandy and Lamport [7], and compensating transactions to Garcia-Molina and Salem's sagas [13]. The **Journal** primitive applies these ideas to LLM agents, where every model call is a nondeterministic event. What Agentical adds is a statement of their soundness inside a probabilistic semantics: replay is not only deterministic but a coupling of the

same randomness, so probabilities computed for the recorded run remain meaningful after recovery (T1).

Record, replay, and resume for LLM agents. Several recent systems record LLM agent runs at their nondeterministic boundaries and replay them, for debugging or regression testing [8, 29]. Khan [19] specifies a checkpoint-and-resume contract for workflow persistence layers, checks it with machine-checked models, and finds duplicate effects after crashes in widely used frameworks; Zheng et al. [43] give an exact, Lean-mechanized check of when agent executions can be checkpointed, forked, restored, or merged; and Wu et al. [41] show that rollback can restore states that are not consistent causal cuts. Li [25] proves that no policy based on verification alone achieves exactly-once effects under late commits without a bound on in-flight time; our effect classes make the corresponding assumptions explicit (external atomicity for Class A, a sound probe for Class C). These works are deterministic or empirical; Agentical places replay and recovery inside a probabilistic semantics.

Durable-execution engines. Temporal [38] re-executes workflow code against an event history and requires that, given the same input, the code issue the same commands in the same order. Calls to the outside world, including LLM calls, run as Activities whose results are recorded in the history, and the SDKs provide replay-safe operations that record other nondeterministic values, such as random numbers [39]. For an LLM agent, each model call therefore becomes an opaque recorded value: the engine makes replay correct but offers no way to state properties of the distribution the value was drawn from. Restate [31], DBOS [10], and LangGraph's checkpointing [23] take related journaling or checkpointing approaches. To our knowledge none of these systems comes with a formal semantics in which replay soundness is a theorem.

8.5 Summary

Probabilistic operational semantics, formal models of LLM agents, information-flow control for concurrent programs, and log-based recovery are each well established. To our knowledge, no prior work combines a probabilistic semantics for concurrently scheduled LLM agents with a journal whose replay soundness is proved within that semantics; that combination is the contribution of this paper.

9 Discussion

9.1 The Rust Prototype

The `agentical` Rust crate has no dependencies beyond the standard library. It defines one synchronous, executor-agnostic trait per primitive (`Agent`, `Oracle`, `Channel`, `Memory`, `Journal`, `Supervisor`); Listing 1 shows three of them.

```
pub trait Agent {
    type State;
    type Out;
    fn step(&mut self, state: &mut Self::State) -> Step<Self::Out>;
}

pub trait Oracle {
    fn sample(&self, prompt: Prompt) -> Sampled;
```

```
    fn measure_hint(&self) -> Option<Dist> { None }
}

pub trait Journal {
    fn append(&mut self, agent: &AgentId, event: Event) -> Cursor;
    fn replay_next(&self, agent: &AgentId) -> Outcome;
    fn checkpoint(&self, agent: &AgentId) -> Cursor;
}
```

Listing 1: Three of the six primitive traits of the `agentical` crate (doc comments omitted).

The journal is per-agent and position-indexed: the cursor is the key, as in assumption (A1), so there is no key-to-value map and no collision assumption. A runtime module implements the parts of the calculus needed for single-agent durable execution:

- `InMemoryJournal`, with `append`, `replay_next`, `rewind`, `truncate` (to simulate a crash), and detection of dangling effect intents;
- `SeededOracle`, a categorical sampler driven by a SplitMix64 generator, which stands in for an LLM;
- a driver that runs an agent in record or replay mode following [O-REC], [O-REP], and [O-RESUME], and performs durable effects with the three-phase write-ahead discipline (intent with argument, perform, committed with result), so that replay returns the journaled result without re-performing the effect;
- recovery of a dangling intent by re-performing it (Class I) or by recording a compensation (Class C).

The `Channel`, `Memory`, and `Supervisor` traits are defined but not yet implemented; in particular there is no multi-agent scheduler, no information-flow enforcement, and no persistent journal backend. The agent used in all experiments is a small two-branch loop (Section 7).

9.2 Limitations

The theorems hold under named assumptions; we list the ones with the most practical weight.

The prototype covers one agent. T1's multi-agent content (send suppression and re-delivery against a consistent cut) and T2 are specified and proved, but the prototype runs a single agent and does not enforce labels. The experiments therefore test single-agent replay and recovery only.

A-CUT: consistent cuts. Crash recovery for concurrent agents (Corollary C1) requires resuming from a consistent cut: all agents in a recovery scope replay together, and a send is suppressed exactly when its matching receive lies below the cut. For tightly coupled agents the only consistent cut may be the whole system, so recovering one agent may force the whole scope to replay. Formalizing finer-grained partial replay remains open.

A-ORC-NI: shared model state. In the calculus an oracle's measure depends only on its prompt. A deployment that serves several labels from one model instance may violate this through shared state such as a key-value cache, a shared system prompt, or server-side memory, and High context could then influence Low outputs. Sound deployment requires isolating model state per label.

A-SCHED and A-TERM: scheduling and termination. T2 holds only for schedulers that cannot observe High steps, and only when High computations terminate almost surely. A scheduler that reacts to how long a secret computation takes (for example, a time-sliced scheduler whose decisions depend on elapsed steps) can leak through timing, as in classic multi-threaded noninterference [32, 40], and a diverging High computation can withhold all later Low events.

A-IND: correlated failures. The product bound of T3 requires independent contract events. Agents that share one LLM endpoint have correlated failures (rate limits, contention), so the product bound can be optimistic; the union bound of T3(a) remains valid. Disjoint channels alone do not give independence.

Effect classes. Recovery is proved for idempotent (I) and atomic (A) effects, holds for compensable (C) effects only if the probe that reports their status is sound (A-EFF-C), and excludes non-recoverable (X) effects (A-EFF-X). Escalating a Class-X fault to a human operator is outside the calculus, and whether that restores a consistent state is the application's responsibility.

Information flow is coarse. The floating-label discipline is conservative: once an agent reads High data it can no longer produce Low output, and the calculus has no construct for deliberate declassification.

9.3 Open Problems

Mechanization. The single-agent core of T1 is mechanized in Lean 4 (`lean/AgenticalT1.lean`, checked with Lean v4.30.0). For a single agent and the oracle fragment, the development models record and replay over a shared outcome stream ω and proves `t1_replay_sound`: replaying a recorded journal reproduces the recorded trace and journal, for every program, every ω, and every starting cursor. It also proves trace equivalence (part (i) of T1), the pointwise identity underlying the coupling of part (iii), the cursor lemma L1, and a pointwise single-agent form of Corollary C1: truncating the journal at any point, replaying the surviving prefix, and re-recording the remainder from the same ω reproduces the uninterrupted run. The development is `sorry`-free, and `#print axioms` reports only Lean's standard axioms (`propext`, and `Quot.sound` for one list lemma). Everything else is on paper: channels, memory, and effects in the replay proof, the measure-theoretic statement of equiprobability, Lemma L4, and T2–T3, whose probabilistic arguments would need a library of measures on infinite paths.

A multi-agent runtime. Implementing channels, a scheduler, label enforcement, consistent-cut recovery, and a persistent journal backend would allow the multi-agent claims (C1 for concurrent agents, T2, T3) to be tested, and would allow experiments with real LLMs.

Session types. Channel interaction types (Section 3) are stated informally. A session-type system [17, 18] with progress and preservation proofs would make the protocol assumptions of T3 checkable.

Distributed journals. The semantics models a single shared journal. A geographically distributed system needs a replicated journal, which introduces consistency questions the calculus does not yet model. T1's argument applies to any journal that satisfies the key-stability and key-distinctness conditions of Lemma 5.1; the open question is which replication protocols preserve them.

10 Conclusion

We have presented Agentical, a probabilistic process calculus for multi-agent LLM systems built on six primitives: **Agent**, **Oracle**, **Chan**, **Mem**, **Journal**, and **Sup**. Each LLM call is a draw from an abstract measure, scheduling is adversarial and kept separate from oracle randomness, and every externally determined value is journaled.

The central result, Theorem T1, shows that replay from the journal reproduces a run with probability 1 and that record and replay are a coupling of the same randomness, including recovery of concurrent agents from a consistent cut. This places a classical recovery technique, logging nondeterministic events, inside a probabilistic semantics, so durability no longer requires treating each model call as an opaque value with no distribution. T2 shows that probabilistic noninterference extends to concurrently scheduled agents under a floating-label discipline and a scheduler that cannot observe secret steps, and T3 gives compositional probability bounds.

The single-agent core of T1 is mechanized in Lean 4, and a Rust prototype implements single-agent record, replay, crash resume, and write-ahead-logged effects; experiments on it confirm replay determinism and recovery from a crash at every journal position.

Future directions. The most important open problems are (1) mechanizing the full T1 and T2–T3, including the measure-theoretic layer; (2) a multi-agent runtime with channels, scheduling, label enforcement, consistent-cut recovery, and a persistent journal, so that the multi-agent results can be tested, including with real LLMs; (3) a session-type system for channels with mechanized progress and preservation; and (4) replicated journals for distributed deployments.

Acknowledgments and Use of AI Tools

I developed this work with Anthropic's Claude as a collaborator on the proofs, code, and writing. Any errors are my own.

References

[1] Isaiah Adams. Noninterference for language models through secure multi-execution, 2026. Preprint.

[2] Gilles Barthe, Thomas Espitau, Benjamin Grégoire, Justin Hsu, Léo Stefanesco, and Pierre-Yves Strub. Relational reasoning via probabilistic coupling. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-20)*, volume 9450 of *Lecture Notes in Computer Science*, pages 387–401. Springer, 2015. doi: 10.1007/978-3-662-48899-7_27.

[3] S. Bensalem, Y. Dong, M. Fränzle, X. Huang, J. Kröger, D. Nickovic, A. Nouri, R. Roy, and C. Wu. Position: A three-layer probabilistic assume-guarantee architecture is structurally required for safe LLM agent deployment, 2026. arXiv:2605.18672.

[4] Benedikt Bollig, Matthias Függer, and Thomas Nowak. Provable coordination for LLM agents via message sequence charts, 2026. arXiv:2604.17612v3; accepted at ISoLA 2026.

[5] Johannes Borgström, Ugo Dal Lago, Andrew D. Gordon, and Marcin Szymczak. A lambda-calculus foundation for universal probabilistic programming. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 33–46. ACM, 2016. doi: 10.1145/2951913.2951942.

[6] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail?, 2025. arXiv:2503.13657.

[7] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985. doi: 10.1145/214451.214456.

[8] Tisha Chawla and Susheem Koul. Chronicle: Cut-point replay for regression testing of LLM agents, 2026. arXiv:2609.20625.

[9] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control, 2025. arXiv:2505.23643.

[10] DBOS, Inc. DBOS Transact. `https://www.dbos.dev`, 2026. Accessed November 2026.

[11] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002. doi: 10.1145/568522.568525.

[12] Zac Garby, Andrew D. Gordon, and David Sands. The LLMbda calculus: AI agents, conversations, and information flow, 2026. arXiv:2602.20064v2.

[13] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259. ACM, 1987. doi: 10.1145/38713.38742.

[14] Jun He and Deying Yu. Post-deterministic distributed systems: A new foundation for trustworthy autonomous infrastructure, 2026. arXiv:2606.01722.

[15] Oltea Mihaela Herescu and Catuscia Palamidessi. Probabilistic asynchronous π-calculus. In *Foundations of Software Science and Computation Structures (FoSSaCS)*, volume 1784 of *Lecture Notes in Computer Science*, pages 146–160. Springer, 2000. doi: 10.1007/3-540-46432-8_10.

[16] Jane Hillston. *A Compositional Approach to Performance Modelling*. Distinguished Dissertations in Computer Science. Cambridge University Press, 1996.

[17] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *European Symposium on Programming (ESOP)*, volume 1381 of *Lecture Notes in Computer Science*, pages 122–138. Springer, 1998. doi: 10.1007/BFb0053567.

[18] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *Principles of Programming Languages (POPL)*, pages 273–284. ACM, 2008. doi: 10.1145/1328438.1328472.

[19] Sajjad Khan. Resume means resume: A machine-checked conformance contract for checkpoint, interrupt, and resume semantics in workflow persistence layers, 2026. arXiv:2608.03836.

[20] Raffi Khatchadourian. Replayable financial agents: A determinism-faithfulness assurance harness for tool-using LLM agents, 2026. arXiv:2601.15322v4.

[21] Dexter Kozen. Semantics of probabilistic programs. *Journal of Computer and System Sciences*, 22(3):328–350, 1981. doi: 10.1016/0022-0000(81)90036-2.

[22] Arseny Kravchenko, Vadim Liventsev, Innokentii Konstantinov, Ildar Iskhakov, and Matvey Kukuy. APPA: Recoverable information-flow control for real-world LLM agents, 2026. arXiv:2607.24625.

[23] LangChain, Inc. LangGraph: Persistence. `https://docs.langchain.com/oss/python/langgraph/persistence`, 2026. Accessed November 2026.

[24] Kim G. Larsen and Arne Skou. Bisimulation through probabilistic testing. *Information and Computation*, 94(1):1–28, 1991. doi: 10.1016/0890-5401(91)90030-6.

[25] Jiapeng Li. Where does exactly-once live? model, harness, and tool-contract effects on duplicate side effects in LLM agents, 2026. arXiv:2609.29095.

[26] Qin Liu. λ_a: A typed lambda calculus for LLM agent composition, 2026. arXiv:2604.11767.

[27] Louis Mandel, Guillaume Baudart, Mandana Vaziri, and Martin Hirzel. PPDL: LLM-based flows as probabilistic programs, 2026. arXiv:2608.05234.

[28] Robin Milner. *Communicating and Mobile Systems: the π-Calculus*. Cambridge University Press, 1999. ISBN 978-0521658690.

[29] Rasheed Mudasiru. Deterministic replay for AI agent systems, 2026. arXiv:2607.16200.

[30] James Peters-Gill, Avi Semler, Henning Bartsch, Ilia Shumailov, and Christian Schroeder de Witt. Can CaMeLs talk? securing multi-agent systems against indirect prompt injection attacks, 2026. arXiv:2610.05640.

[31] Restate. Restate. `https://restate.dev`, 2026. Accessed November 2026.

[32] Andrei Sabelfeld and David Sands. Probabilistic noninterference for multi-threaded programs. In *Proceedings of the 13th IEEE Computer Security Foundations Workshop (CSFW)*, pages 200–214. IEEE Computer Society, 2000. doi: 10.1109/CSFW.2000.856937.

[33] Andreas Schlapbach. Formal semantics for agentic tool protocols: A process calculus approach, 2026. arXiv:2603.24747v3.

[34] Roberto Segala and Nancy Lynch. Probabilistic simulations for probabilistic processes. In *CONCUR '94: Concurrency Theory*, volume 836 of *Lecture Notes in Computer Science*, pages 481–496. Springer, 1994. doi: 10.1007/978-3-540-48654-1_35.

[35] Alaia Solko-Breslin, Pramod Kaushik Mudrakarta, Mihai Christodorescu, Somesh Jha, and Krishnamurthy Dj Dvijotham. Efficient and sound probabilistic verification for AI agents, 2026. arXiv:2606.20510.

[36] Deian Stefan, Alejandro Russo, John C. Mitchell, and David Mazières. Flexible dynamic information flow control in Haskell. In *Proceedings of the 4th ACM Symposium on Haskell*, pages 95–106. ACM, 2011. doi: 10.1145/2034675.2034688.

[37] Robert E. Strom and Shaula Yemini. Optimistic recovery in distributed systems. *ACM Transactions on Computer Systems*, 3(3):204–226, 1985. doi: 10.1145/3959.3962.

[38] Temporal Technologies, Inc. Temporal workflow definition. `https://docs.temporal.io/workflow-definition`, 2026. Accessed November 2026.

[39] Temporal Technologies, Inc. Side effects (Go SDK). `https://docs.temporal.io/develop/go/workflows/side-effects`, 2026. Accessed November 2026.

[40] Dennis Volpano and Geoffrey Smith. Probabilistic noninterference in a concurrent language. *Journal of Computer Security*, 7(2–3):231–253, 1999. doi: 10.3233/JCS-1999-72-305.

[41] Guanlong Wu, Dahui Li, Ke Jiang, Jianyu Niu, Cong Wang, and Yinqian Zhang. Safe to resume? breaking execution continuity of agent execution via rollback, 2026. arXiv:2608.29381.

[42] Sofiya Zaichyk. Semantic fusion: Verifiable alignment in decentralized multi-agent systems. *ACM Transactions on Autonomous and Adaptive Systems*, 21(3):1–29, 2026. doi: 10.1145/3797042.

[43] Yusheng Zheng, Xiaoyu Song, Yanpeng Hu, Lebin Cheng, Yuxi Huang, and Wei Zhang. When can agents safely checkpoint, fork, restore, and merge? exact checking for execution edits, 2026. arXiv:2608.22928.

[44] Timothy Zhou, Loris D'Antoni, and Nadia Polikarpova. Language-based agent control, 2026. arXiv:2605.12863.

A Notation

This appendix collects the notation used throughout the paper.

Table 7: Canonical notation for the Agentical calculus.

Glyph	Meaning	Introduced
Syntactic categories		
v, w	Values: closed, irreducible results	§4.1
e	Expressions: pure, deterministic computations	§4.1
M, N	Terms (processes): agent behaviour	§4.1
A	Agent: $id[M]_{\Sigma}$	§4.1
S	System: parallel pool of agents	§4.1
c, d	Channel names (π-calculus)	§4.1
k	Memory key / cell identifier	§4.1
$\ell \in (\mathcal{L}, \sqsubseteq)$	IFC label in lattice; $\mathsf{Low} \sqsubseteq \mathsf{High}$	§4.1
$\rho \in \{\mathsf{turn}, \mathsf{sess}, \mathsf{dur}\}$	Memory retention	§4.1
$id \in AID$	Agent identity	§4.1
IFC lattice		
$(\mathcal{L}, \sqsubseteq)$	Label lattice with partial order	§4.1
Low	Public/low label	§4.1
High	Secret/high label	§4.1
$\sqsubseteq$	"No more secret than"	throughout
$\sqcup$	Label join (l.u.b.)	§3.6

Table 7 (continued)

Glyph	Meaning	Introduced
$Lo \subseteq \mathcal{L}$	Observable label set (downward closed, closed under joins)	§5.3
$pc(\Sigma)$	Agent's current (floating) label	§4.5
ℓ_c	Label of channel c	§4.2
Oracle measure		
μ_e	Oracle distribution for prompt e; μ_e a probability measure on Val	§3.4
μ_p	Oracle distribution after $e \Downarrow p$	§4.5
$\mathcal{D}$	Sub-probability distribution over successor configs	§4.5
$\delta_{C'}$	Dirac (point-mass) on C'	§4.5
Journal (J) and events		
J	Journal: finite append-only event sequence	§4.2
ε	Empty journal	§4.2
$J \cdot e$	Append event e to J	§4.2
$\mathsf{key} \in \mathcal{K}$	Journal key: stable step identifier	§4.2
$\mathsf{key}(C, id)$	Key of agent id's next journaled step	§4.2
$\mathsf{append}(J, e)$	Journal append	§4.2
$\mathsf{lookup}(J, k)$	Lookup event by key; returns event or $\bot$	§4.2
$\mathsf{OracleDraw}(k, v, prov)$	Oracle-draw event	§4.2
$\mathsf{ChannelRecv}(k, v)$	Channel-receive event	§4.2
$\mathsf{ChannelSend}(k, v)$	Channel-send event	§4.2
$\mathsf{Effect}(k, f, v, s)$	Durable-effect event; $s \in \{\mathsf{intent}, \mathsf{committed}, \mathsf{compensating}\}$	§4.2
$\mathsf{Clock}(k, t)$	Clock-read event	§4.2
$J \restriction id$	Journal projection for agent id	§4.5
Cursor	Durable checkpoint (journal length at a durable point)	§3.7
$\mathsf{WL}(J, R)$	J is well-labelled w.r.t. reduction sequence R	§4.7
Configuration		
$\langle P;\, H;\, J;\, K;\, t \rangle$	System configuration 5-tuple	§4.2
P	Agent pool: multiset of $id[M]_\Sigma$	§4.2
H	Shared labeled store: $k \rightharpoonup (\ell, \sigma, \rho, v)$	§4.2
K	Channel state: queues + restricted names	§4.2
Σ	Private agent memory	§4.2
$t \in \mathbb{N}$	Logical clock (monotone, durable)	§4.2
Modes		
REC	Record mode (live execution)	§4.5
REP	Replay mode (crash recovery)	§4.5
Schedulers		
$\mathfrak{s}$	A scheduler resolving adversarial nondeterminism	§4.6
$\mathfrak{S}$	Set of all schedulers	§4.6
$\mathsf{enabled}(C)$	Enabled transitions at C	§4.6
Reduction relations and judgments		
$C \xrightarrow{a}_m \mathcal{D}$	Probabilistic step: label a, mode m, dist. $\mathcal{D}$	§4.5
$e \Downarrow v$	Big-step pure evaluation	§4.1
$\equiv$	Structural congruence	§4.3
$\approx$	Observational trace equivalence	§4.4
Transition labels		
τ	Silent / local	§4.5
$o(k, \ell)$	Oracle draw at key k with output label ℓ	§4.5
$c\,!\,v$	Send v on c	§4.5

Table 7 (continued)

Glyph	Meaning	Introduced
$c\,?\,v$	Receive v on c	§4.5
$\iota(k, \ell)$	Memory read at key k, label ℓ	§4.5
$\omega(k, \ell)$	Memory write at key k, label ℓ	§4.5
$\mathsf{eff}(k, f)$	Durable effect f at key k	§4.5
$\mathsf{spw}(id')$	Spawn agent id'	§4.5
$\lightning$	Fault event	§4.5
Trace measure and probability		
$\mathbb{P}_{\mathfrak{s},C_0}$	Scheduler-induced probability measure on paths	§4.6
$\mathbb{P}^{\mathrm{REC}}_{\mathfrak{s},C_0}$	Trace measure in record mode	§5.2
$\mathbb{P}^{\mathrm{REP}}_{\mathfrak{s},C_0}$	Trace measure in replay mode	§5.2
$\Pr[\cdot]$	Generic probability	throughout
$\mathbb{E}[\cdot]$	Expectation	§5.4
$\hat{\mathfrak{s}}$	Coupled scheduler (T1 proof)	§5.2
$\mathrm{obs}(C)$	Observable view of C	§4.4
Noninterference (T2)		
$C_0 =_{Lo} C_0'$	Low-equivalence of configurations	§5.3
tr_{Lo}	Low trace: the subsequence of Low-visible labels	§5.3
Agent contracts (T3)		
$A \models_p \varphi$	Agent satisfies φ w.p. $\geq p$ (all schedulers)	§3.3
$1 - \sum_i (1 - p_i)$	Union bound (no independence)	§5.4
$\prod_{i=1}^{N} p_i$	Product bound (requires A-IND)	§5.4
Fault and supervisor		
$\lightning$ ($\lightning$)	Fault tag (generic)	§4.5
$\mathsf{restart}$	Decision: replay from journal	§3.8
$\mathsf{compensate}(saga)$	Decision: run compensating saga	§3.8
$\mathsf{escalate}$	Decision: re-raise to parent	§3.8
stop	Decision: abandon	§3.8
$\mathsf{replay}(N,\, J \restriction id)$	Re-enter N in replay mode	§4.5